

Ph.D Thesis Introductory Chapter

Yi Gu

April 22, 2025

Contents

1	Overview	1
2	t-SNE and Dimension Reduction	2
2.1	The Ubiquity and Challenge of High-Dimensional Data	2
2.2	A Brief History of Dimensionality Reduction	6
2.3	The Road to t-SNE	12
3	Markov Random Field and Probabilistic Graphical Models (PGM)	16
3.1	Dependency: An Unavoidable Problem	16
3.2	PGM, a Short History	20
3.3	Bayesian Networks and Markov Random Fields	25
3.4	The Necessity of Sampling and Lovász Local Lemma	30
4	Factor Analysis and Variational Inference	32
4.1	Combining DR and PGM	32
4.2	Lineage of Factor Analysis	33
4.3	The Modern Toolkit: Variational Autoencoders	34
4.4	The High-Level View of VarFA based on VAE	39

1 Overview

As we venture further into the 21st century, our world is shaped, built, or dare we say, defined by data (Hey et al. [2009]). From simple information like class rosters, to large scale datasets like the Human Genome Project, we are recording everything, with the realization that there are limitless values to be extracted. Mathematics and algorithms are at the center of this grand picture, as the extent to which we can unlock this value is fundamentally constrained by our mathematical understanding. Indeed, for decades have we collected visual data such as satellite imagery and medical scans, but the studies and utilization of them remained primarily manual. In 2012, the success of ImageNet(Krizhevsky et al. [2012]), along with

Convolution Neural Network (CNN), made automated analysis of visual data possible. It was moments like this that revealed to us there is so much to be seen and discovered with data.

Among all the mathematical theories and tools that forge the foundation of data science, probability theory and probabilistic algorithms are of unique and paramount importance. With data comes uncertainty and noise, and with forecasting and prediction comes ambiguity. The nature of the tasks data science aims to tackle and the intrinsic problems stemming from all forms of data call for probabilistic approaches. They provide the mathematical language and framework for us to navigate through randomness with deterministic confidence.

Take the example of CNN we just mentioned. The success of CNN is seen by many as the watershed moment in the history of mathematics and computer science. Though the advancements in semiconductor industry and hardware engineering should never be overlooked, the birth of CNN is the result of a series of brilliant developments in probability since Kholmogorov. The applications of concepts like softmax layers, maximum log-likelihood estimation, etc all have a deep root in probability theory and constitute the core mechanism of CNN (Krizhevsky et al. [2012]). And our stories have just begun. As of today, the development of Large Language Models (LLM), the closest ever creation of humans to Artificial General Intelligence (AGI) is essentially a probabilistic algorithm (Vaswani et al. [2017]).

In this thesis, we will explore three different probabilistic algorithms. In chapter 2, we will study t-SNE, another famous and important dimension reduction algorithm developed by Hinton (van der Maaten and Hinton [2008]). We will show that we are able to swap out the widely used input and output kernels and still achieve convergence and equilibrium measure in the asymptotic case. In chapter 3, we will transform a classic theorem, Lovász Local Lemma, into a probabilistic algorithm that can learn difficult combinatorial structures. We will also provide the theoretical foundation of the algorithm, proving its correctness and giving an estimation of the running time expectation. In chapter 4, we will dive into another dimension reduction algorithm, Variational Factor Analysis (VarFA), and how we can combine the classic Variational AutoEncoder (VAE) with factor analysis to create a more interpretable generative model. All three algorithms represent a different yet extremely important aspect of data science, each with its own charm and characteristics.

For the rest of this chapter, we will provide a brief introduction to the probabilistic algorithms we will discuss in this thesis. From basic concepts, to history, and related progress. It is very exciting to see all the mathematical concepts our predecessors have developed, and have been passionate about, are being turned into new technologies that impact and steer the modern world. And may this thesis provides an insignificant, yet meaningful addition to this great painting of our future.

2 t-SNE and Dimension Reduction

2.1 The Ubiquity and Challenge of High-Dimensional Data

Most data and data sets we deal with on a daily basis are multi-dimensional by nature. For example, a simple class roster can have multiple exam and homework scores. If the total number of assignments is greater than 3, then this single class roster is already beyond what we can natively visualize (if we want to represent each student as a point). On the other hand, each image can be seen as a high-dimensional vector and the dimensionalities increase rapidly while the resolution we are able to capture advances with technology. The number of variables scale in a breakneck pace as we are more capable of capturing detail-rich data as well as automating the process (Donoho [2000]). As stated in the last section, this process

gives us practically unlimited potentials. The objects and processes we would like to study were recorded with utmost details, unlike the past when vital bits of information could be lost forever. However, this also brings caveats and new mathematical challenges.

One of the most important and hardest challenges is the noise that comes with data. Apart from noise caused by engineering factors, the growth of dimensionality also brings noise because many dimensions may be irrelevant for the problems one want to solve. The extra dimensions themselves become the “noise”. In 1996, Robert Tibshirani discovered the famous Least Absolute Shrinkage and Selection Operator (LASSO) for the purpose of variable selection (Tibshirani [1996]). Consider a sample consisting of N cases, each with dimension p . Let y_i be the outcome and x_i be the p -dimensional vector that represents the i -th case. LASSO aims to solve

$$\min_{\beta_0, \beta} \sum_{i=1}^N (y_i - \beta_0 - x_i^T \beta)^2, \quad (2.1)$$

with constraint

$$\sum_{j=1}^p |\beta_j| \leq t. \quad (2.2)$$

LASSO marks the early realization and effort mathematicians and scientists were trying to mitigate the overwhelming information brought by high dimensional data. LASSO was also closely related to Basis Pursuit Denoising (BPDN), introduced in 1994 (Chen and Donoho [1994]). BPDN is a well known NP-hard problem. Before the age of big data comes, mathematicians had been thinking about extracting useful signals from noise/spurious correlations. After the dawn of 21st century and the explosive growth of computer science, problems like BPDN and LASSO become even more important after we add computational cost to the equation (Fan and Lv [2010]).

Apart from feature and variable selection, people also seek to “compress” the data, creating a mathematical summary of the essence. t-SNE is one of such examples. Instead of making a selection, t-SNE aims to find a low-dimensional representation of the data and taking the entire data set into account. Another good example is the Variational Auto Encoder (Kingma and Welling [2014]), a generative model that is able to scale well with large data sets using a low-dimensional “latent space”. We will discuss VAE and our related work on this topic in chapter 4.

In the face of large amount of high dimensional data, digesting it “as is” is more often than not impossible. The above examples show us that we either make a selection, or we perform compression, which are the two most common approaches in terms of getting what we need from the data. We would like to discuss this further because high-dimensional data is not only mathematically and computationally hard to deal with, but also tends to mask the insights we are looking for. This is somehow a paradoxical nature of high-dimensionality. Faithful recording of data assures us the information and values we seek are within the data sets and within in our reach. But the form and scaling of the high-dimensional data also buries the light we are after. This is the so-called “curse of dimensionality”.

The term “curse of dimensionality” was first coined by Richard Bellman in 1961 (Bellman [1961]). It refers to various phenomena that appears when analyzing data in high-dimensional space that do not occur when dimensionality is low. Qualitatively speaking, the “curse” usually means that as dimensionality increases, the exponential increase of volume makes the same amount of data sparse and harder to work with.

To understand this concept, let us first look at the famous example of hypercube vs hypersphere. Consider a hypercube with edge length $2r$ in d -dimensional Euclidean space and its inscribed sphere with radius r . Denote Γ the gamma function and C_d , S_d the volume of the

hypercube and hypersphere respectively. Then we have

$$\lim_{d \rightarrow \infty} \frac{S_d}{C_d} = \lim_{d \rightarrow \infty} \frac{\pi^{d/2}}{d 2^{d-1} \Gamma(d/2)} = 0. \quad (2.3)$$

This means that our low-dimensional intuition fails. The volume of hypercubes are mostly “concentrated” in the corners and the inscribed sphere’s volume becomes negligible. If we want to analyze the data related to the hypercube, a data set that is uniformly distributed in the hypercube will become an increasingly small part of the whole space. In general, as we increase the dimension of our sample space, any finite data set will occupy a vanishingly small part of it. On the other hand, to maintain the roughly same density, one needs to accumulate exponentially more data points to compensate for the increase in dimensions.

Such fact also induces a data mining problem, where it is unrealistic to gather enough amount of data points to guarantee the separability of data generated by two different distributions. It is a well known fact in machine learning that too many features with inadequate data points will lead to overfitting. This discovery is not new, either. In 1968, Hughes showed that adding more features past certain points without providing more data points will cause worse classification performance (Hughes [1968]). Later it becomes known as the famous Hughes phenomenon. Adding features is not a free lunch and the results could be detrimental.

To further illustrate the curse of dimensionality, let us look at two extra examples, one mathematical and one from the field of biology. We just compared the volume of hypercubes and hyperspheres as the dimensionality tends to infinity. This comparison is closely related to another phenomenon called the concentration of distance. It refers to the fact that uniformly sampled points in high-dimensional space tend to be equidistant from each other. This can be seen more intuitively from the two collections of plots below.

For the second plot, we replace the unit cube with the unit sphere as our sample space. On the top of distance concentration, we can see that most points are located towards the surface of the hypersphere, rather than the interior. This aligns with our intuition that the volume of the hypersphere is mostly concentrated near the surface, and the volume of a hypercube is mostly concentrated near the corners.

The above two plots reinforce the intuition behind the data mining problem we just brought up. Distance, one of the most fundamental and important feature that distinguishes data points, becomes less and less effective as the dimension of the data increases. It directly leads to the fact that purely distance-based algorithms will perform poorly with high-dimensional data sets. It is crucial to find ways to extract the information we need from high dimensions, then compress, analyze and visualize it in lower-dimensions. The concentration of distance is also one of the reasons that algorithms such as t-SNE are invented. As we will see in the coming sections, t-SNE encodes the neighborhood information using probability distributions, rather than plain Euclidean distance.

The last example we would like to share is the RNA sequencing and many genome projects, where the curse of dimensionality manifests itself in real-world scientific research. Generally speaking, RNA sequencing experiments measure the abundance of tens of thousands of different RNA transcripts on expression level. Such rich information can exist simultaneously within a single biological sample because life itself is tremendously complex. From a data science point of view, all these genes/transcripts are features and each acts as a dimension so the dimensionality is extremely high. For humans, that number is somewhere roughly between 10^4 to 10^5 . For Genome sequencing that number can be even larger, ranging from 10^6 to 10^7 .

This causes all kinds of problems we mentioned about high-dimensional data (Stegle et al. [2015], Kiselev et al. [2019], Visscher et al. [2017]). On one hand, Euclidean distance becomes less meaningful. The Euclidean distance between two patients (data points) may not contain any useful implications for their similarities in genetic traits or tendencies for a specific

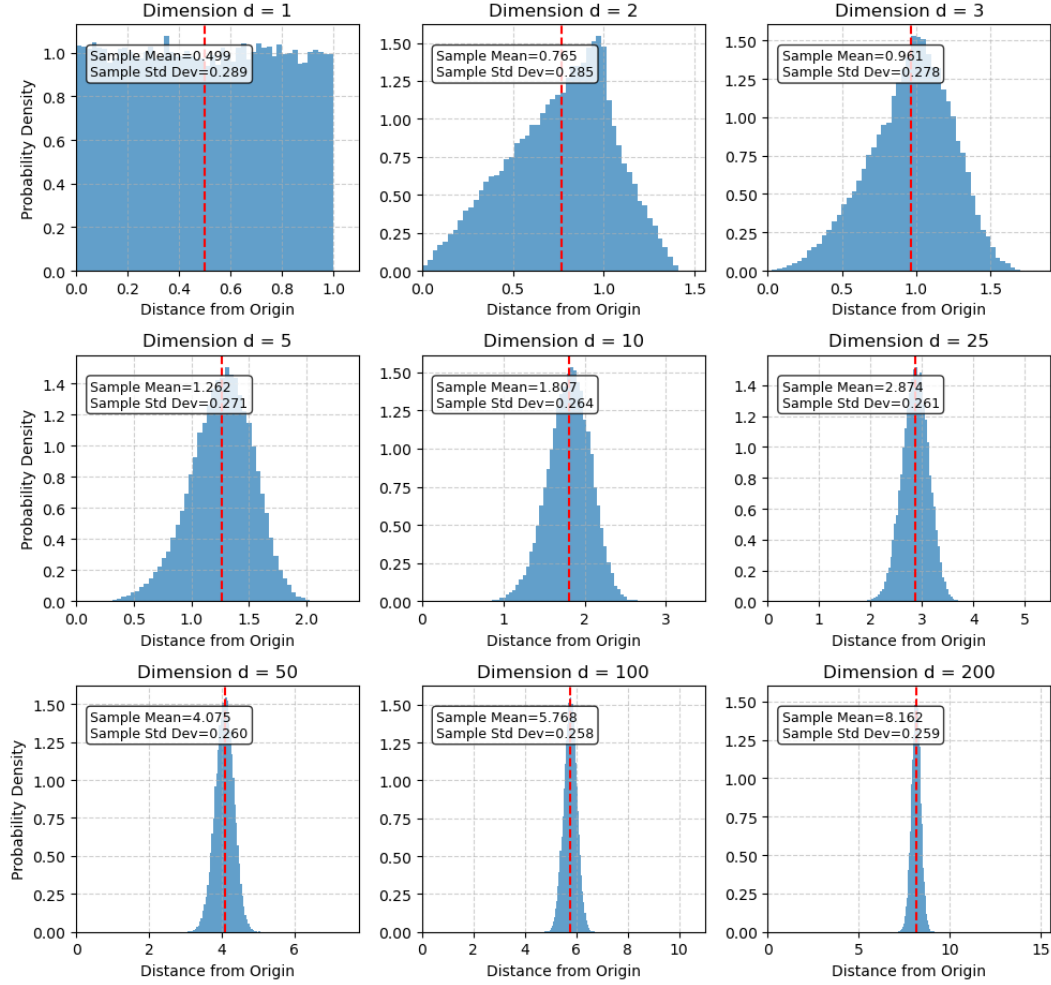


Figure 1: The distribution of distance from points uniformly sampled in unit cubes of different dimensions. The vertical red lines indicate the average distance between points. It is clear that as the dimensionality grows, the distribution of distance becomes more concentrated.

disease. The occurrence of spurious correlations are much higher, that people often finds significant correlations between features(genes) and outcomes(disease status) that are purely coincidental and not biologically meaningful. On the other hand, Hughes phenomenon is prevalent in model training because it is nearly impossible to gather enough data so that the sample size is significantly larger than the number of features. This results in massive overfitting and poor predictions for critical problems such as tumor types classification. At the same time, genetic sequences are absolute nightmares for hypothesis testing due to their sheer complexity, making false positives much more prone to happen.

For these very reasons, dimension reduction really shines through the field of biology and bioinformatics. Most algorithms introduced in the next section are still widely used today, from the old classic PCA, to methods created in 21st century such as t-SNE. Dimension reduction is a crucial toolset that makes continuing research and analysis of genetic data possible, with its impact and usage expanding well beyond the field of biology to almost all scientific subjects related to data science, statistics and machine learning.

With all the above being said, it is of paramount importance to study dimension reduction

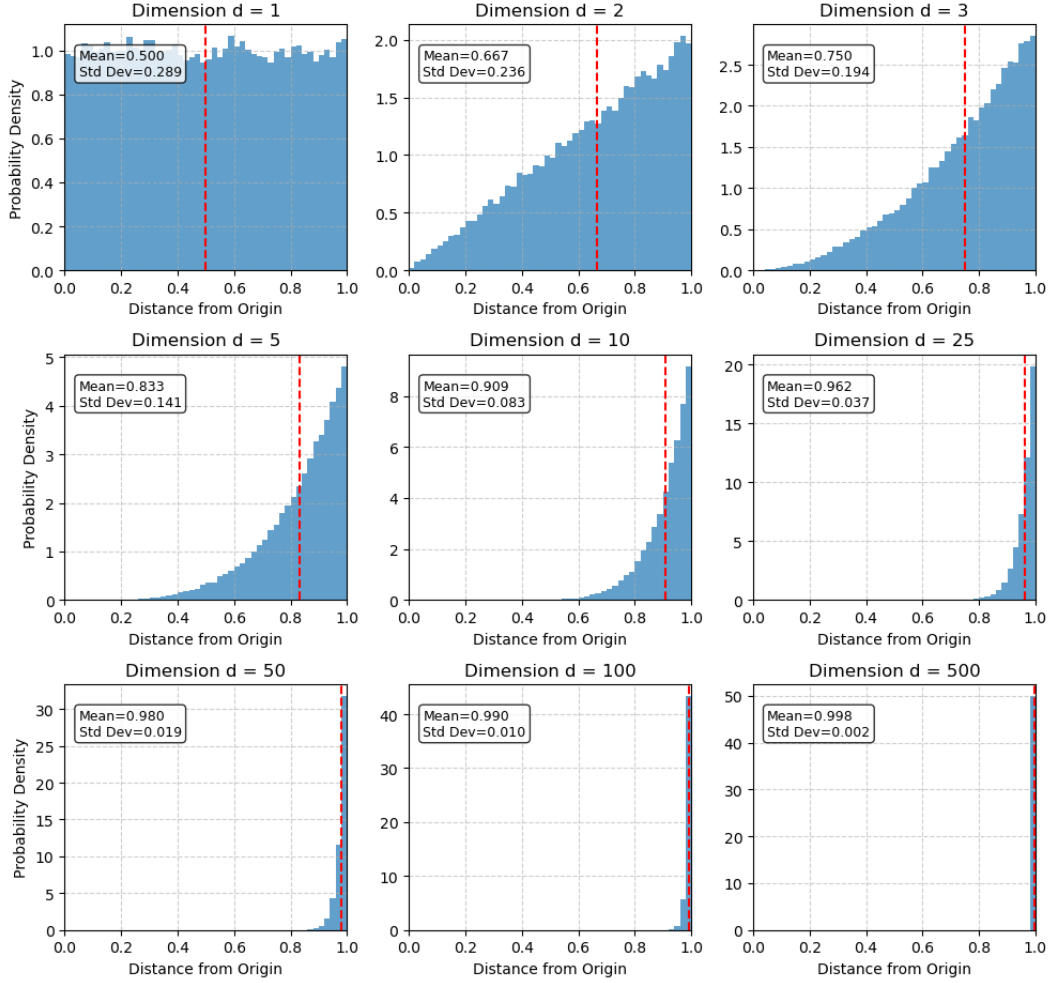


Figure 2: The distribution of distance from points uniformly sampled in unit spheres of different dimensions. The vertical red lines indicate the average distance between points. It is clear that as the dimensionality grows, the distribution of distance concentrates towards the length of radius.

techniques and algorithms. The knowledge of which aspects of data are important is obviously not a priori, so it is impossible to select the features/dimensions we need beforehand. The only option for us is to develop the mathematical theory, and reduce the dimensionality to the extent that the information we seek becomes apparent. Now we are almost a quarter into the 21st century, the development of technology resonates with the need for dimension reduction, and puts our mathematical legacy to the test.

2.2 A Brief History of Dimensionality Reduction

The theory of dimension reduction can be traced back to the beginning of 20th century. In 1901, Karl Pearson introduced the geometric idea of finding lines and planes of best fit to data points, later known as the famous Principle Component Analysis (PCA) (Karl Pearson [1901]). This technique was also independently discovered by Harold Hotelling in 1933 (Hotelling [1933]). The idea behind PCA is mainly linear algebra. It first centers the data set at zero and then tries to find the largest eigenvalue of the covariance matrix. The corresponding eigenvector marks the direction of the most variance in the data, making it

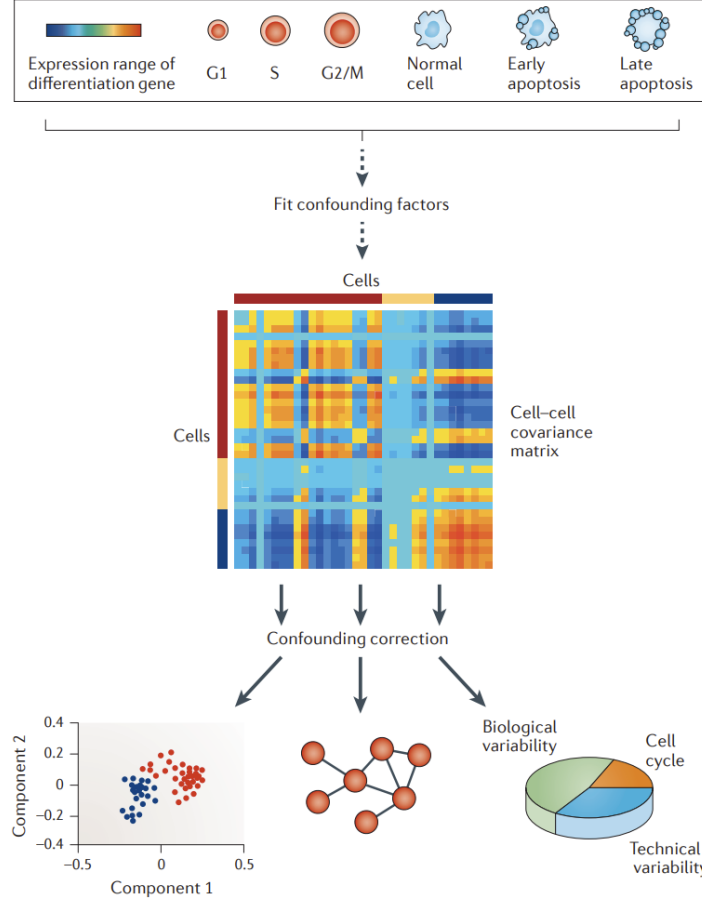


Figure 3: Part of data processing pipelines introduced in [Stegle et al. \[2015\]](#), with dimension reduction (PCA) being one of the key steps (covariance matrix and visualization).

the biggest principle component. Then this component gets subtracted and the algorithm repeats to find the next largest eigenvalue. The algorithm can stop at any number of principle component. The corresponding eigenvalues and eigenvectors will then be used to construct a subspace with lower dimensions that represents the original data.

Intuitively speaking, PCA tries to fit a high-dimensional ellipsoid to the data set. Each axis of the ellipsoid represents one principle component. The larger the eigenvalue, the longer the corresponding axis of the ellipsoid. PCA is simple and elegant, so much so that it is still widely used today, but its shortcomings are also apparent. PCA assumes linearity as what it does is primarily finding a new coordinate system where the axes, a.k.a. the principle components, are linear combinations of the original variables. The mapping from the original high-dimensional space to the lower-dimensional subspace defined by principle components is an orthogonal linear transformation. This means that by analyzing the covariance matrix, PCA can mostly only capture linear dependencies. This approach will fail if the intrinsic structure of the data is non-linear. For example, PCA is likely to give two principle components if asked to analyze a circle, a typical 1-dimensional manifold in 2-dimensional Euclidean space. It is unable to see that the points on a circle are all part of a 1-dimensional space. Similar results with Swiss roll, PCA will very likely be unable to “unroll” the Swiss roll into its natural 2-dimensional representation, but rather cut through it.

PCA is one of the most important and widely used dimension reduction algorithms. Though

its assumption on linearity limits its application, it also allows it to be computationally efficient using modern hardware and parallelism. It outperforms many modern algorithms in terms of speed and scalability. For this reason PCA is included in many modern software packages like scikit-learn and PyTorch.

Continuing onward, in the 1950s, Warren Torgerson invented Multidimensional Scaling (MDS) (Torgerson [1952], Torgerson [1958]), with the classical version often known as Torgerson Scaling. MDS is one of the early non-linear dimension reduction algorithms. Given a distance/dissimilarity matrix (where the distance does not have to be Euclidean distance) and a target dimension N , MDS aims to find a set of points in N -dimensional space where the pairwise distances are preserved as much as possible. Often one lets $N = 2$ so the result can be easily visualized. In classical MDS, the final goal is to minimize the loss function, which is also called strain:

$$\text{Strain}(x_1, \dots, x_n) = \left(\frac{\sum_{i,j} (b_{ij} - x_i^T x_j)^2}{\sum_{i,j} b_{i,j}^2} \right). \quad (2.4)$$

Here, x_i 's are vectors in N -dimensional space that we are trying to find, and b_{ij} are the entries from the Gram matrix that can be computed using the input data. Compared to PCA, MDS no longer assumes linear relationships between data points, and is more intuitive as the distance between points is often a good reflection of their similarity. One of the most famous applications of MDS is the color wheel. One might never hear about MDS, but they must have used the color wheel at some point of their lives.



Figure 4: The famous 2-dimensional color wheel, created by MDS. It effectively and intuitively captured the similarity and discrepancies between different colors on the spectrum of visible light.

Being intuitive and flexible, MDS gained a lot of popularity and it was developed into a family of algorithms. Shepard and Kruskal developed non-metric MDS independently in 1962 and 1964 (Shepard [1962], Kruskal [1964]). Their work allowed MDS to handle Ordinal data, meaning the modified algorithm only requires the rank order of dissimilarities. Such outstanding flexibility is one of the major reasons that it is still widely used today, especially when there is a lack of good distance metrics or quantitative data, such as market research. However, MDS is not without drawbacks. The axes of the output do not have inherent meanings from the original data set and sometimes it is not possible to find a solution with low enough stress values (the stain in the classical MDS). More importantly, MDS is not computationally efficient. Though modern algorithms like t-SNE and UMAP share some of the disadvantages with the MDS family, they often outperform MDS by a fair margin.

Nevertheless, the limitations of MDS algorithms do not diminish their importance. MDS is still one of the most important algorithms in the field of dimension reduction.

Fast forwarding to the millennium, the rapid advancement of computer science and applied mathematics created increasingly stronger needs to study data of nonlinear nature. Scientists had been striving to create new tools and algorithms that can handle the emerging complex data sets. Though mentioned in earlier works, many regarded the work of Roweis and Saul in 2000 as the pillar of manifold learning (Roweis and Saul [2000]). Unlike PCA, which tries to project a collection of high dimensional vectors onto a linear subspace, manifold learning projects the high-dimensional data onto a lower-dimensional latent manifold. Since manifold learning was created specifically to tackle problems with nonlinear nature, it is also called Nonlinear Dimension Reduction (NLDR).

In the same paper, Roweis and Saul presented the Local Linear Embedding (LLE), one of the classic algorithms in NLDR. From this point onward, readers who are familiar with SNE and t-SNE may notice the resemblance between the algorithms and how they draw inspiration from each other. Given original data points $X_i \in \mathbb{R}^D$, the goal of LLE is to find a set of points $Y_i \in \mathbb{R}^d$ usually with $D \gg d$ such that Y_i 's form a good representation (embedding) of X_i 's. This is done in two steps. First, LLE looks at each point X_i and tries to find the closest approximation of X_i using the linear combination of other points. i.e., for each i , LLE computes

$$\left\| X_i - \sum_j W_{ij} X_j \right\|^2, \quad (2.5)$$

where W_{ij} are the weight we assigned to each X_j . W_{ij} is nonzero when X_j is within a certain distance of X_i , or when X_j is considered a "neighbor". This distance is a hyperparameter set by the user. Summing up through all i 's gives us the reconstruction error:

$$E(W) := \sum_{i=1}^N \left\| X_i - \sum_j W_{ij} X_j \right\|^2. \quad (2.6)$$

While LLE aims to minimize $E(W)$, it is also subject to the restriction

$$\sum_j W_{ij} = 1. \quad (2.7)$$

This ensures that W is not affected by rotation and translation. The first step is finished when LLE successfully finds the weight that minimizes $E(W)$. This is a typical constraint least square problem so we are able to find the global minimum. Now the weight W_{ij} 's are fixed and the second step begins. LLE now tries to find output Y_i 's that minimize the following cost function:

$$C(Y) := \sum_{i=1}^N \left\| Y_i - \sum_j W_{ij} Y_j \right\|^2. \quad (2.8)$$

The algorithm terminates when the minima of $C(Y)$ is found. Directly from the same paper (Roweis and Saul [2000]), we know that finding the minimum of $C(Y)$ is equivalent to solving a sparse eigenvalue problem and we are able to find the global minimum. This is one of the most appealing mathematical properties of LLE, that both of its core steps yield globally optimal solutions. However, nothing is perfect and LLE does come with its own shortcomings. Let us discuss that after we introduce another NLDR algorithm that was created around the same time, and is just as famous and important: Isomap.

Isomap was first introduced in around 2000 (Tenenbaum et al. [2000]). It is a modern generalization of MDS. Instead of using the Euclidean distance, Isomap generates the so called geodesic distance between each pair data points then apply MDS to get the low-dimensional output. We start with our high-dimensional input $X_i \in \mathbb{R}^D$. The first step of Isomap is to find the neighbors of each X_i . This can be achieved by either considering a ball with radius ϵ around X_i and see which points are inside the ball, or by using the k nearest neighbors. Both ϵ and k are hyperparameters set by the user.

After getting the neighbors for each point, we will construct a weighted graph G based on the information we just obtained. Each vertex of G represents an input data point. We will use the term vertex i when the vertex represents X_i . We draw an edge between vertex i and vertex j if X_j is a neighbor of X_i , which is determined during the previous step of the algorithm. The weight of the edge (i, j) is the Euclidean distance between X_i and X_j . Next up, we perform the Dijkstra algorithm on G , which yields the shortest graph distance between each pair of vertices i and j . This distance is called the geodesic distance, denoted by $d_G(i, j)$. This concludes the second step of Isomap.

Afterwards, since $d_G(i, j)$ is well defined for each tuple (i, j) , we have a distance matrix as the input of the MDS algorithm. Running the algorithm with the target low dimension d will produce the final result of Isomap.

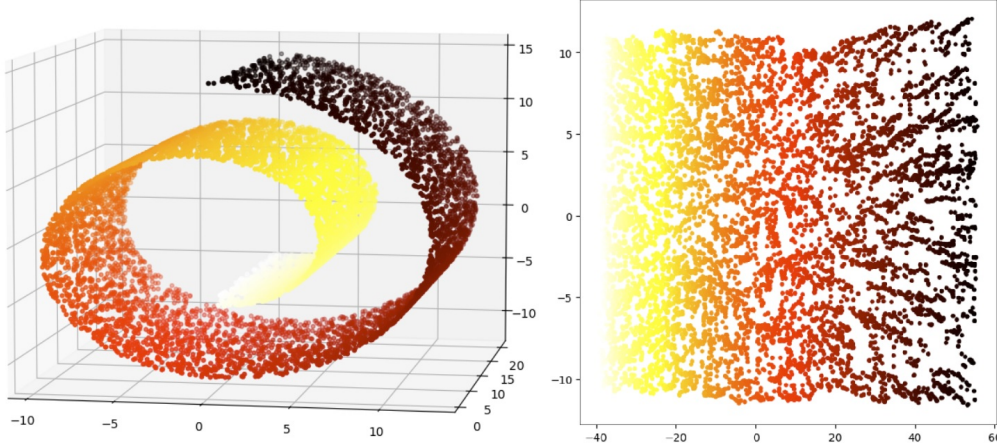


Figure 5: The input Swiss roll (left) and the output of Isomap (right). The Isomap successfully captures the nature of the manifold and “unrolls” the Swiss roll.

With the introduction of Isomap, it is easier to explain the drawbacks of LLE. As we can see in the steps of LLE, it emphasizes the local reconstruction of each data point using its neighbors. Because of this, LLE can potentially fail to preserve the global structure of the data, namely the distances on a larger scale. Points that are very far away in the original data set can be mapped close to one another in the output of LLE, especially when the space itself is very “curved”. This shortcoming is addressed by Isomap, as the intermediate step of finding the graph G is specifically designed to preserve global structure. Using MDS subsequently ensures that such preservation is translated into the low-dimensional output. The graph G represents the grand picture Isomap tries to summarize. Nevertheless, Isomap is not perfect, either. As we already mentioned earlier in this section, MDS is a computationally expensive algorithm. On top of that, given a graph $G = (V, E)$, the Dijkstra algorithm has a time complexity of $O(|E| + |V| \log |V|)$, where $|E|$ and $|V|$ are the number of edges and vertices in the graph respectively (Fredman and Tarjan [1984]). For dense graphs, $|E|$ will scale quadratically with $|V|$. By combining two computationally intense algorithms, Isomap is inevitably slow and becomes a bad choice for large and dense

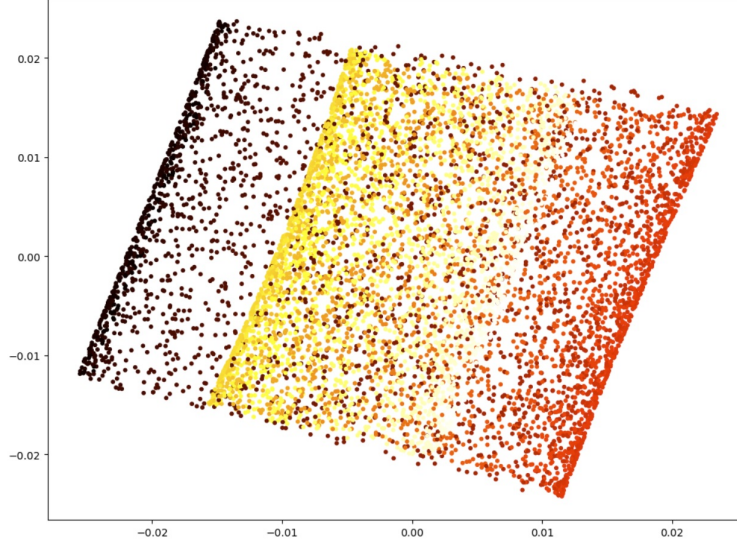


Figure 6: A failed LLE output based on the same Swiss roll input in figure 5. The algorithm tries to search for neighbors in an overly broad area, causing the result to be distorted.

data sets.

Moreover, Isomap is very sensitive to the choice of hyperparameters and outliers. If the number of neighbors or the ball radius is too small, Isomap will sometimes produce a disconnected graph, prevents the algorithm from generating a global structure. But if the hyperparameters are too large, the algorithm might include “short-circuit” edges that connect points across gaps or folds in the manifold that are close in the ambient Euclidean space, such as distant points on a tightly packed Swiss roll. Then in this case Isomap will contain the worst of both worlds, by being an extraordinarily slow algorithm that also carries the disadvantages of LLE. Besides that, since Isomap is designed to preserve the global structure, it will cover everything, including outliers. Having an outlier in the data set, such as a point far away from everything else, can significantly disrupt the output.

Finally, Isomap carries some implicit assumptions. It assumes that the data on the manifold is well sampled, meaning that the graph constructed in Isomap is a truthful representation of the true geodesic distances on the manifold, from a differential geometry perspective. It also assumes that the intrinsic geometry of the manifold can be isometrically embedded into a lower-dimensional Euclidean space, hence the name. Such assumptions are not always true, and we should treat the output with caution if the stress values (similar to the definition in (2.4)) are high.

Both LLE and Isomap are important milestones in the advancement of dimension reduction algorithms. With different focusing aspects, both provided us with a useful set of tools to analyze high-dimensional data, suited for different scenarios. However, mathematicians and computer scientists did not stop there. All the algorithms up to this point still work primarily with Euclidean distance. What if we want to extract the abstract structure of similarity out of the data sets, rather than the distance itself? What if we encode the information of neighborhood using probability distribution, rather than plain physical proximity? Such ideas inspired the creation of stochastic neighbor embedding (SNE) and the subsequent t-stochastic neighbor embedding (t-SNE). And that is where our story begins.

2.3 The Road to t-SNE

SNE was first introduced in 2002 by Hinton and Roweis ([Hinton and Roweis \[2002\]](#), with Roweis also being the creator of LLE). Like we just mentioned at the end of the previous section, SNE uses probability distributions to record the information of neighborhood. With this new framework, we need a new concept to measure the distance between two probability distributions, and that is the famous Kullback-Leibler (KL) divergence.

Definition 1. *Let P and Q be two probability measures with $p(x)$ and $q(x)$ as their respective probability density functions. The KL divergence between P and Q is defined as:*

$$D_{KL}(P||Q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) dx. \quad (2.9)$$

In particular, if P and Q are discrete probability measures(distributions) defined on the same sample space $\mathcal{X}$, then the KL divergence is defined as:

$$D_{KL}(P||Q) = \sum_{x \in \mathcal{X}} p(x) \log \left(\frac{p(x)}{q(x)} \right). \quad (2.10)$$

Note that the KL divergence is zero if and only if P and Q are the same distribution. At the same time, it is not symmetric and is not a true distance metric. It is also called the relative entropy. Since both SNE and t-SNE will construct probability distributions from the original data set, rather than plain Euclidean distance, KL divergence is used to formulate the loss function we aim to minimize.

SNE starts with defining the following probability mass function for each data point X_i : for each $j \neq i$, let

$$p_{j|i} = \frac{\exp(-\|X_i - X_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|X_i - X_k\|^2 / 2\sigma_i^2)}. \quad (2.11)$$

σ_i is a hyperparameter that is set by the user or by computation. We restrict that for each i , the induced probability distribution has the same entropy for all i . This means for some constant ρ , we have

$$\sum_{j \neq i} p_{j|i} \log(p_{j|i}) = \log(N\rho), \quad (2.12)$$

where N is the number of data points. The quantity $N\rho := \text{Perp}$ is called perplexity. This is a very clever design and it serves as a “slide bar” for each X_i . For example, the majority of the data points are located inside a ball with radius 1 and there is one data point that is more than 1000 units away. Then from that point’s perspective, the rest of the data points are roughly “equally” distant from itself. The induced probability distribution will be very close to a discrete uniform distribution. A lot of information is lost during this process. By introducing perplexity, we allow each induced probability distribution to zoom in and out so that they are “observing” the rest from the data set on the same scale. Later we will see that this setting is not only backed up by clear intuition, but also plays an important role in proving the convergence to equilibrium measure in chapter 2, as well as in the prior work by Auffinger and Fletcher ([Auffinger and Fletcher \[2023\]](#)).

Let Y_i be the low-dimensional output of SNE. We can define a similar probability mass function:

$$q_{i|j} = \frac{\exp(-\|Y_i - Y_j\|^2)}{\sum_{k \neq i} \exp(-\|Y_i - Y_k\|^2)}. \quad (2.13)$$

Next, SNE will then try to minimize the KL divergence between the two (collections of) distributions, i.e.:

$$L = \sum_{i=1}^N D_{KL}(P_i || Q_i) = \sum_{i=1}^N \sum_{j \neq i} p_{j|i} \log \left(\frac{p_{j|i}}{q_{j|i}} \right). \quad (2.14)$$

Since the input X_i 's are given and fixed, the loss function L is a function of the output points Y_i 's. The minimum can be found using gradient descent. Different from LLE, SNE is not guaranteed to find a global minimum.

Though the formulation of SNE resembles LLE, the invention of SNE was still a refreshing evolution. From plain distance, to geodesic distance from constructed graphs, and now to probability distribution, it is good to see that mathematicians are using more flexible concepts to encode the general landscape of data. But people soon discovered that SNE has its own problems. Since the output distribution decays exponentially and it does not have a slide bar like perplexity, SNE is not able to distinguish clusters that are not in close proximity. This is the well-known ‘‘crowding problem’’ of SNE, and it directly inspired the birth the subsequent t-SNE algorithm ([van der Maaten and Hinton \[2008\]](#)).

The t-SNE algorithm shares the same procedure as SNE. However, instead of using the Gaussian function in the output schema, t-SNE uses the much slower decaying student's t-distribution with 1 degree of freedom, i.e.,

$$q_{ij} = \frac{(1 + \|Y_i - Y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|Y_i - Y_k\|^2)^{-1}}. \quad (2.15)$$

That is also where the letter ‘‘t’’ in the name comes from. The t-distribution greatly mitigates the crowding problem, and it is able to perform very well across different data sets. Also, t-SNE defines the input probability mass p_{ij} as

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}, \quad (2.16)$$

which makes the input probability masses symmetric. One of the greatest and most well-known showcase of t-SNE is its implementation on the MNIST data set. The Modified National Institute of Standards and Technology (MNIST) is a large database consisting of handwritten digits. It is one of the mostly commonly used public data set for machine learning and computer vision. Started with USPS database that records handwritten zip codes, today's MNIST contains 60000 training images and 10000 testing images and serves as a benchmark for many machine learning algorithms. t-SNE is able to effectively capture the structure of data and clearly cluster different digits into separate groups. This algorithm is one of the greatest achievements in the field of dimension reduction and unsupervised learning in the 21st century. Unfortunately, we need to repeat the same statement again: nothing is perfect. t-SNE is also a computationally expensive algorithm. If the data set is large and complex, it is advised to pre-process the data set using lightweight dimension reductions such as PCA. Meanwhile, though with better flexibility and overall performance, t-SNE inherits the same drawback from LLE: poor performance at preserving global structures. The distance between two clusters are often meaningless. We only know that the corresponding original data points do not belong to the same group, but little information can be extracted by the distance between the resulting low-dimensional clusters. t-SNE is also non-deterministic by nature: ending results might vary slightly for the same data set and hyperparameters. While the general cluster structures might be similar, the exact positions and orientations can vary, which can be sometimes confusing.

Such traits of t-SNE inspired some discussion regarding the mathematical nature of the algorithm. First, if t-SNE is non-deterministic, can we say something about its convergence and asymptotic behavior? Will the output of t-SNE converge to a certain equilibrium

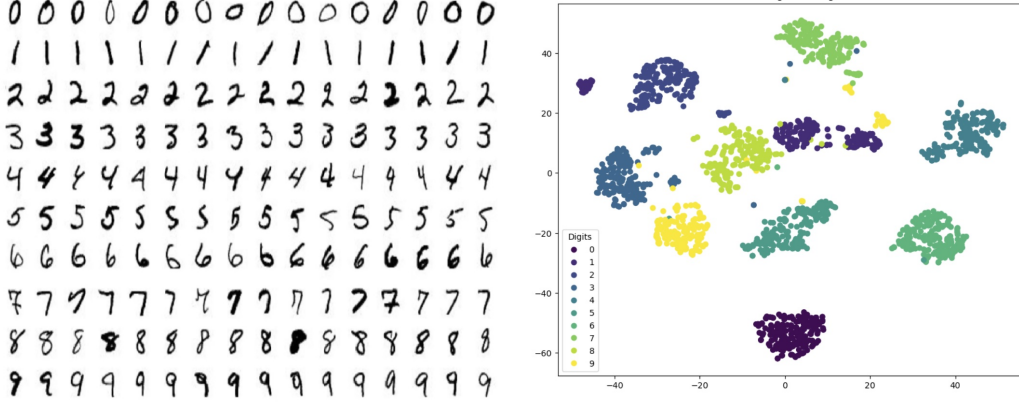


Figure 7: A sample of the MNIST dataset (left) and the output of t-SNE (right). t-SNE successfully grouped the handwriting with different digits together, with some minor imperfections (there are two clusters for the digit 0).

measure as we increase the number of samples to infinity? The answer is yes. We have the following results from [Auffinger and Fletcher \[2023\]](#):

Theorem 1 (Theorem 1.1, [Auffinger and Fletcher \[2023\]](#)). *Assume all input data points of t-SNE are drawn independent from a probability measure μ_X . μ_X has a C^1 density function $f(x)$ and compact support. Denote X, Y the input and output data set, n the number of data points, ρ the perplexity, and $L_{n,\rho}(X, Y)$ the loss function (KL-divergence) of t-SNE.*

Then for any $\rho \in (0, 1)$ given, there exists a function I_ρ and a set of measures $\mathcal{P}_{\mu_X}$, such that

$$\lim_{n \rightarrow \infty} \inf_Y L_{n,\rho}(X, Y) = \inf_{\mu \in \mathcal{P}_{\mu_X}} I_\rho(\mu). \quad (2.17)$$

If we write

$$\mu_n = \frac{1}{n} \sum_{i=1}^n \delta_{X_i, Y_i} \quad (2.18)$$

as the empirical measure of the input and output data set, then the following theorem also holds:

Theorem 2 (Theorem 1.2, [Auffinger and Fletcher \[2023\]](#)). *Assume the same conditions as in theorem 1. Then there exists a measure μ^* and a sub-sequence $\{n_k\}$ such that μ_{n_k} converges weakly to μ^* , and $\mu^* = \inf_{\mu \in \mathcal{P}_{\mu_X}} I_\rho(\mu)$. Moreover, μ^* has compact support.*

The formulation of the theorems are slightly different from the original paper and some technical details are omitted due to the introductory nature of this chapter. The above two theorems send a strong message about the theoretical guarantee of t-SNE. If the input data is drawn independently from the same distribution, t-SNE is able to output a global minimum equilibrium measure (here measure means the distribution of the output data set) as we increase the sample size to infinity, with compact support. These are the first rigorous mathematical results about this algorithm after it has been used for so many years. And it leads to the next question, the question we are able to address in chapter 2.

For the output distributions, using the Gaussian is not necessary as the successful execution of the algorithm does not rely on the specific properties of Gaussian functions. The two theorems above further verify this observation. This gives rise to the concept of kernels.

For a pair of points (x, y) , a kernel is a positive function of $\|x - y\|$ that we can use to determine the normalizing weights that produce the input and output distributions. For input, the kernel is the Gaussian function and for the output, the kernel is the function $f(x) = (1 + x^2)^{-1}$. It is natural to ask:

“Is the student t-distribution not the only kernel that might give us the same theoretical guarantees? If no, then what is the class of output kernels that will give us the same results in [Auffinger and Fletcher \[2023\]](#)? Even more generally speaking, if we swap out both input and output kernels, what conditions do we need for the same theorems to be true?”

These are the exact questions we are able to answer with rigorous proofs in chapter 2. Besides that, people are already swapping out output kernels in practice. Note that in the formula of KL-divergence, for the term $p_{ij} \log(p_{ij}/q_{ij})$ reaches its minimum zero when $p_{ij} = q_{ij}$. This means if we manually increase p_{ij} , the gradient descent process will tend to increase q_{ij} as well. This is achieved by putting the output points Y_i and Y_j closer together, as the output kernel $f(x) = (1 + x^2)^{-1}$ is monotone decreasing. Therefore, in application people often replace p_{ij} with its multiple kp_{ij} for some constant $k > 1$. This is called the early exaggeration, one of the frequently used techniques. Using early exaggeration will result in closer clusters, improving visualization results.

Another known modification of the t-SNE algorithm is the heavy-tailed t-SNE. Recall that the density function of the t-distribution is given by

$$f(x) = \left(1 + \frac{x^2}{\alpha}\right)^{-\frac{\alpha+1}{2}}, \quad (2.19)$$

where α is the degree of freedom. Decreasing α will reduce its decaying speed and make it more “heavy-tailed”. This will make q_{ij} relatively larger and the output clusters will be compact. It is also shown in [Kobak et al. \[2020\]](#) that heavy-tailed t-SNE tends to produce finer clusters, such as sub-clusters of the same digit in the MNIST data set. Both early

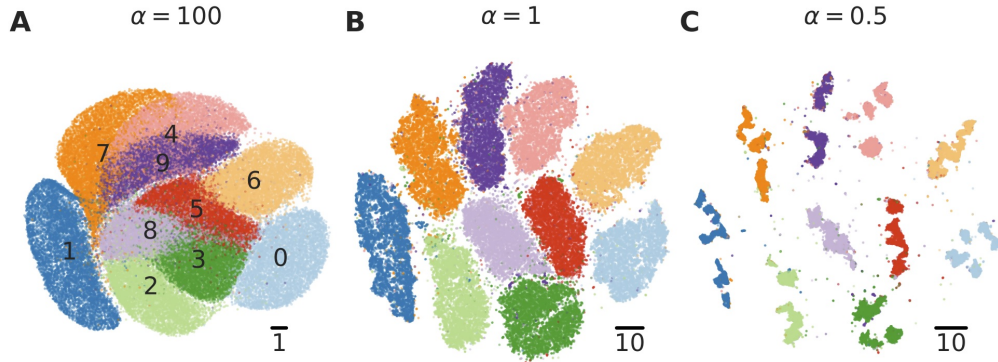


Figure 8: The running result of different SNE algorithms in [Kobak et al. \[2020\]](#), with (A) being the light-tail t-SNE, (B) the normal t-SNE and (C) the heavy-tailed t-SNE. Heavy tailed variants are producing tighter and finer clusters, while the light-tail version produces results similar to the original SNE. (t-distribution will converge to Gaussian if $\alpha \rightarrow \infty$.)

exaggeration and heavy-tailed t-SNE can be seen as manipulations of the output kernel. t-SNE works effectively under the modification and produces promising results, besides different characteristics researchers hope to see. Besides exploring different conditions for the convergence to hold and the equilibrium measure to exist, it is also an interesting problem to see how exactly the output changes with the modification of the output kernel. Some insightful results can be found in [Weinkove \[2024\]](#).

It is fascinating to see the rigorous mathematical theory behind widely used algorithms like t-SNE, which are often used to produce qualitative rather than quantitative results. Getting to the bottom of the convergence behavior will provide valuable insights on the algorithm itself, refining the theoretical landscape of the dimension reduction theory. We will resume our journey in chapter 2.

3 Markov Random Field and Probabilistic Graphical Models (PGM)

3.1 Dependency: An Unavoidable Problem

We have seen how the theory of dimension reduction has come a long way for the purpose of addressing the challenges from high-dimensional data. Now we turn to another, likely more fundamental problem that probability, statistics and data science need to face: dependency. Though most of probability textbook starts with the concept of independence and theorems assuming it, dependencies are probably one of the oldest and most prevalent challenges we have had to tackle ever since the concept of probability was born. How farmers might change their planting plans based on weather; how gamblers leverage their bets based on the cards in hand; how traders design their investment strategies based on the stock market. All these decisions are based on dependencies and uncertainty. Probabilistic dependency has been with us since the beginning of our time. Even to this day, people might still care about the fate of Schrodinger’s cat, “condition on” the mysterious nature of quantum superposition.

The explosive advancement of computer science and modern industry not only brings high-dimensional data, but also exponentially more complicated dependencies that come with it. Let us take a look at several examples¹ that demonstrate the difficulties and complexity of such problems. The first instance is image segmentation and the famous paper written by Geman and Geman ([Geman and Geman \[1984\]](#)). With the introduction of digital imagery, we receive a grid of many pixels, as well as a grid of intricate dependencies. The color and intensity of any given pixel is highly likely to be similar to that of its immediate neighbors. This is true for almost all real images that include actual content. Pixels from the same object they depict tend to share common characteristics, and that is how our eyes capture and process the world we see through various coherence. If we treat all pixels as independent noises, we will be getting static noise that does not convey any information. Therefore, the authors treat the problem of image segmentation and denoising as a statistical inference problem, with Markov random field being the main tool they deployed to encode the information.

This paper is one of the most famous and important paper in the field of computer vision and pattern analysis. It utilizes a lot of important tools and concepts like Bayesian inference, Gibbs sampling, and Markov random field, some of which will be introduced more systematically in the next several sections. The paper also draws inspiration from statistical physics and today the intersection between statistical physics, math, and computer science has developed into a even larger and more captivating field. It is easy to write a couple of books just based on the keywords from this single paper so we will not go into too much nuances. Rather, let us talk about some of the real-life impact this discovery brought.

One of the major application of image segmentation is self-driving cars. In order to identify various objects on the road and navigate through ever-changing traffic, self-driving cars need to have cameras and computer vision technologies to detect and identify objects. This is the aspect that cameras can be much more useful than radars (while radars are better

¹All images used in this section are under free or open copyright licenses such as CC BY-SA and CC BY. All images have been properly credited.

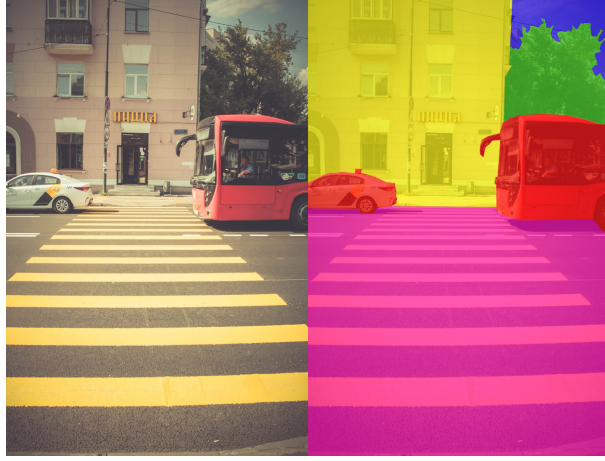


Figure 9: A simple example of image segmentation, by B Palac, from Wikimedia Commons. Different objects like the bus, the road and the buildings in the background can be detected and identified by algorithms. This technology is widely used in critical fields like self-driving cars.

handling positional and distance information). In 2025, we already have commercialized self-driving taxi services by Waymo in several major cities like San Francisco, Los Angeles, and Phoenix, with some known for its notoriously difficult traffic conditions. Not only doing business, Waymo is also active in related research fields, and some of its published results still draw inspiration from Geman and Geman in 1984 (Zhu et al. [2023]).

Another well-known and important follow-up is the Graph Cuts, emerging in early 2000s (Boykov and Jolly [2001], Kolmogorov and Zabini [2004], Boykov and Kolmogorov [2004]). This new image segmentation algorithm was solid step up from Geman and Geman's work and it profoundly influenced the world. Due to its massive improvement in computational efficiency and overall performance, it is widely used in many areas like medical imaging. MRI and CT are able to identify organs, lesions and tissues with the help of segmentation and it is safe to say these devices saved countless lives. Graph Cuts also contributed to the development of stereo vision, which allows the depth of images to be recovered by using two or more cameras, similar to how our eyes work. The narratives and stories above can all be traced back to 1984, where the dependencies problems were effectively modelled and addressed. Vision is the most common sense we use to perceive the world. It contains great obstacles just as it contains great values.

Another good example of dependency research is again, not surprisingly, about genetic sequencing. One of the core reasons that genetic research is extremely difficult is the dependencies between different genes. They do not work in isolation. The expression level of one gene can activate or inhibit the expression of others such as the lac Operon and the trp Operon in *E. coli* (Naville and Gautheret [2010], Jacob and Monod [1961]). For humans a well known genetic piece is the Estrogen Receptor ($ER\alpha$). $ER\alpha$ plays a vital role in the development and function of many tissues, particularly in the female reproduction system, but also in bone, brain and cardiovascular health. $ER\alpha$ affects our body primarily by interacting and binding with other DNA sequences, which ultimately leads to changes in cell behavior. Particularly, $ER\alpha$ dysregulation is a key driver in the majority of breast cancers (Frasor et al. [2004], Heldring et al. [2007]). Drugs that block estrogen binding or $ER\alpha$ activity are critical therapies for breast cancer treatments.

This is just one graph of the vast nest of dependencies in our genetic sequence. Therefore scientists felt it imperative to use mathematical tools to encode, analyze and understand the dependency relationship. Gene Regulatory Networks (GRN) is one of such creations. GRNs



Figure 10: A Waymo self-driving taxi, operating in downtown San Francisco, by Dllu, from Wikimedia Commons. One can easily spot the giant central hub for sensors on the top of the car, as well as many other additional cameras and radars the taxi relies on to deliver a smooth transportation experience. Waymo uses a mixture of technologies, with computer vision and image segmentation being one of the key component of its smooth operation.

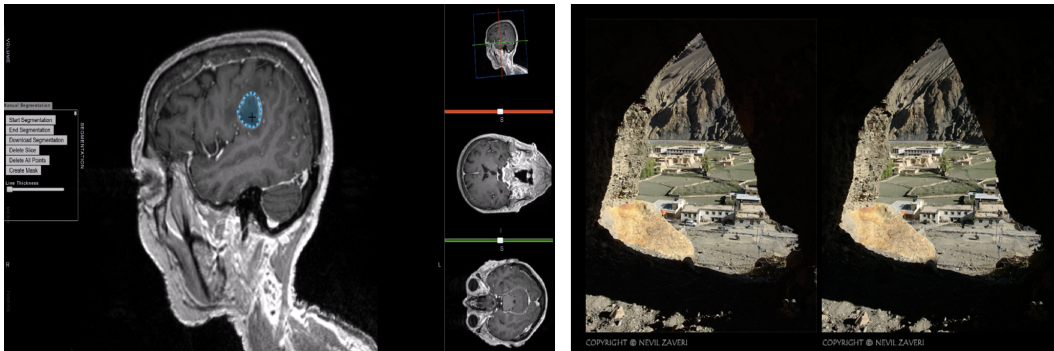


Figure 11: A medical image segmentation that identifies the tumor, by EmmaWiki2020 from Wikimedia Commons, and a 3D image(s) of Heritage Tabo Gompa by Nevil Zaveri from Flickr. To see the 3D image on the right, focus on the center line of the image pair. A third image will appear, the stereo one.

use Bayesian Networks (we will cover more details in section 3.3.) to model the relations between genetics and cellular function, development and disease. By representing genes as nodes and regulatory influences as directed edges, scientists can infer plausible probabilistic relationships between gene expression levels, even in the presence of noise (Friedman et al. [2000], Sachs et al. [2005]). GRNs are often better understood visually than through textual explanation. Please see figure 12 to see how five different types of cancers are related to a large collection of genes via different types of regulatory influences. Without the help of probability and Bayesian network, such information is impossible to be extracted.

While GRNs map interactions between genes, Hidden Markov Models (HMM) aim to decipher the functional grammar within a single sequence (Krogh [1994-01-31], Durbin et al. [1998]). It is another extremely important tool in genetic research like GRN but with a different focus. One key concept HMM has that is not present in GRN is called the hidden states. Generally speaking, the observed gene sequence is just the tip of the iceberg. There are so many active and biochemical reactions that happen before a segment of gene is formed and observed. To give an instance, for eukaryotic genes we have concepts of exon

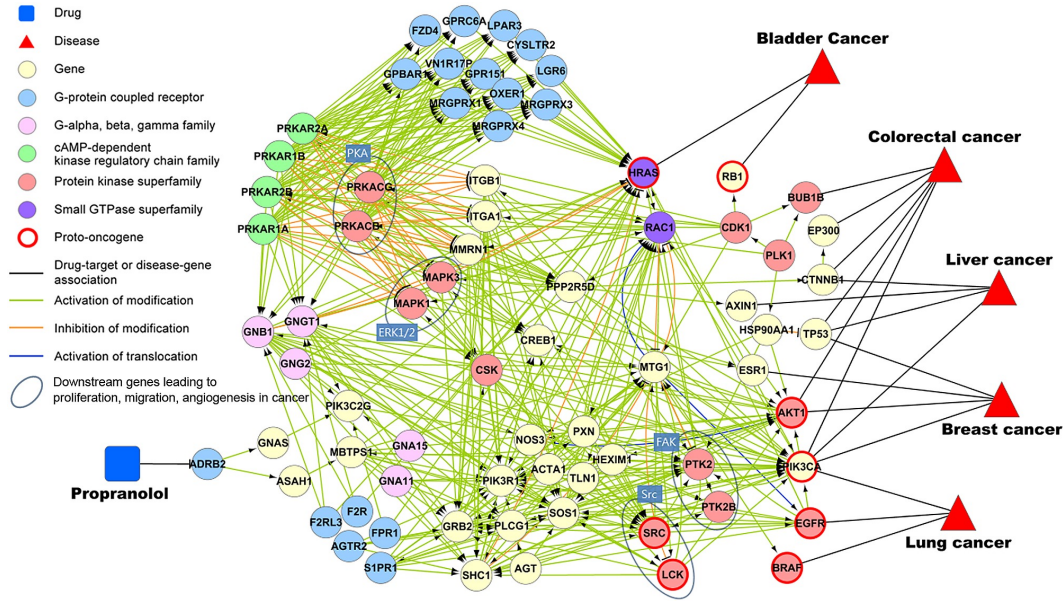


Figure 12: An example of GRN, by Min Oh, Jaegyeon Ahn, and Youngmi Yoon, from Wikimedia Commons. Graph structure is crucial in visualizing complex relationships between genes and different types of cancers.

and intron. Exons are the coding regions that are present in the final mRNA transcript, while introns are the non-coding regions that get removed during RNA splicing. One of the key functions of HMM is to identify the hidden states, the status “behind the scene”, that connect an intron state to a possible exon state, using a framework similar to the Markov chain, hence the name. For example, if we have a hidden state with some particular intron, HMM will find the transition probability of this state to other hidden states, and ultimately, to different observed sequences.

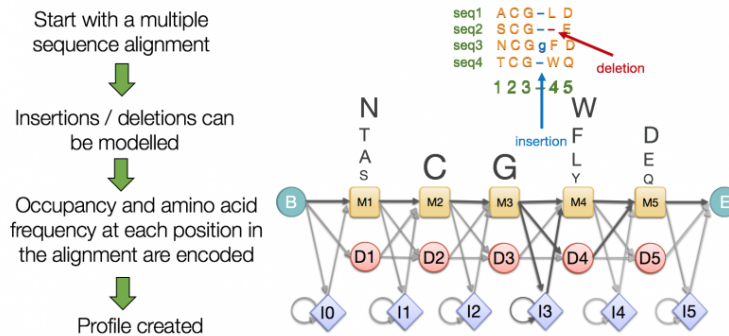


Figure 13: A illustration about HMM modeling multiple sequence alignment, by EMBL-EBI Train Online, from Wikimedia Commons.

Contrary to GRNs, whose goal is often the network structure itself, HMMs are used for classification and alignment, such as calculating the probability that a given sequence was generated by a particular collections of hidden states. We call this the emission probabilities. If we see a coding regions with a certain sequence of nucleotide A, C, G, T or some types of amino acids, we are likely able to predict the likelihood of obtaining a specific observable symbol. HMM reveals and decodes the mechanism behind gene formation, allowing us to

understand the process of gene expression and regulation on a systematic level.

For both examples we just went through (image segmentation and genetic sequencing), we can see the incredible values contained within dependencies, because dependencies encode relations. It is relations that define the world we live in, not isolated objects. All the applications in those examples benefit from properly modelled dependencies. And all those dependencies are transcribed in graph structures. Indeed, probabilistic graphical models are specifically designed to address the dependencies problems, and there have been so many successful advancements and applications of PGMs. As we venture deeper into the finer structure of this universe, both on micro and macro scales, PGMs have become even more important than ever before.

Remark 1. *Though PGMs are the specialized tool to solve dependency problems, we should also be aware that they are not the only tools. As vast as the theories and applications of PGMs have come to be, they are a small part of a much larger overarching theme, combinatorial optimization (CO). This term can refer to any problems that aim to find an optimal object from a finite set of objects (Schrijver [2003]), often comes with a set of constraints as well. Probabilistic dependency is only one kind of constraints and PGMs are only one type of tools that record such constraints. The field of CO is extremely diverse and rich. We can start with basic examples like the traveling salesman problem all the way to the major unsolved problems such as $P=NP$. It is easy to go too deep into the rabbit hole. Therefore, it would be appropriate to acknowledge the wider picture while focusing on PGMs in this section.*

3.2 PGM, a Short History

Both graph theory and probability have a long history. Though the foundation of modern probability is widely regarded as the work by Kolmogorov, probability theory, in general, can date back to the 16th century (LIGHTNER [1991]). Graph theory is often believed to be the brain child of Euler (Euler [1736]). The basic ingredients of PGM have existed long before PGM itself was created. Therefore, the history of PGM can be roughly divided into three parts: the early conceptual seeds (from probability and graph theory), the emergence of mainstream models (Bayesian networks and Markov random fields), and the unification/expansion era that came with the advent of modern computer science.

Prior to the 1980s, there was not a definitive concept of PGM that was widely acknowledged and used among the research community. However, the practices that used graphs to represent probabilistic models started to emerge in the first half of the 20th century. In 1925, Ernst Ising used the lattice graph in his Ph. D thesis to represent the the magnetic dipole moments of atomic “spins” (Ising [1925]). Later this idea was generalized by Lars Onsager in 1944 (Onsager [1944]) and then evolved into the famous Ising model we know today in statistical physics. Formally speaking, let Λ be the lattice graph(site) and we use positive integers to denote its vertices. For each vertex(site) k , there is a discrete variable σ_k such that $\sigma_k \in \{-1, 1\}$, representing the site’s spin. We call the collection of all spins $\sigma := \{\sigma_k\}_{k \in \Lambda}$ the spin configuration. For any two adjacent sites(vertices) $i, j \in \Lambda$, there is an interaction J_{ij} . Finally, let μ be the magnet moment and h_j be the external magnetic field interacting with j . The Ising model is defined by the following random function called the Hamiltonian:

$$H(\sigma) = - \sum_{\langle ij \rangle} J_{ij} \sigma_i \sigma_j - \mu \sum_{j \in \Lambda} h_j \sigma_j, \quad (3.1)$$

where the first summation is over all pairs of vertices/sites i, j that are adjacent on the lattice. Ising model is widely considered the first model in the study of spin glasses. The Hamiltonian effectively captures the interaction, thus dependencies between each spin via a

graph structure. Exchanging the lattice graph with other types of graph will give us some of the other famous models, such as the Sherrington-Kirkpatrick model (complete graph) and Edwards-Anderson model (high-dimensional lattice). Though by definition spin glass models can be seen as PGMs, the studies about them do not have a strong relation with tools in graph theory. In fact, it is a drastically different field compared to the topic we are discussing in chapter 3. Rather, we mention Ising model to highlight the novel representation of mathematical objects that bring probability and graph theory together.

Another early example is path analysis introduced by Sewall Wright in 1921 (Wright [1921], Wright [1934]). This method was created to understand and quantify the relationships among a set of correlated random variables, based on a hypothesized causal structure. Wright represented the hypothesized causal relationship using path diagrams (essentially Directed Acyclic Graphs or DAGs), where nodes represent variables and arrows represent direct causal influences.

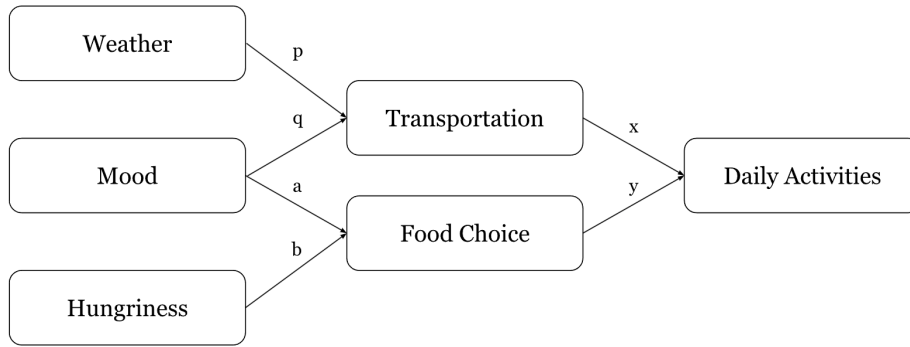


Figure 14: An intuitive example of path diagrams with edge weights (contributing factors). The daily activities of a person depend on the transportation and food choices, which further depend on the weather, mood and how hungry the person is. Using the diagram, we can conduct analysis such as estimating the impact of weather on the person’s daily activities.

With the help of directed graphs, complicated chain of influence can be decomposed and quantitatively studied. The correlation/causation between two variables are measured by the sum of all paths that connect them and the contribution of a path is determined via the product of all its edge weight. This simple but effective framework allows us to estimate the magnitude of direct and indirect effects within the causal model.

Wright himself was also a population geneticist and he used path analysis extensively to study inheritance. Some of his famous work included the piebald patterns of Guinea-Pigs and factor analysis for offspring phenotype (Wright [1920], Wright [1921]). In fact, Wright had already started using the concept of path analysis in 1918 before he formally defined the term (Wright [1918]). It is safe to say the his biological research inspired the creation of this important mathematical model, yet another great victory for the interdisciplinary research between math and biology!

It is also worth noting that the Wright’s work has profoundly influenced the field of PGM and computer science. Path analysis is considered a direct precursor to modern Structural Equation Modeling (SEM). In 1960s and 1970s, social scientist used path analysis and created a set of tools that are still widely used today in social science, psychology and economics (Duncan [1975], Bollen [1989]).

The inspiration of path analysis did not stop there. Fast-forwarding to the 1980s, the concept of PGM was widely recognized and used with the invention of Bayesian networks and Markov random fields, the two pillars of the realm. Continuing from Wright’s work,

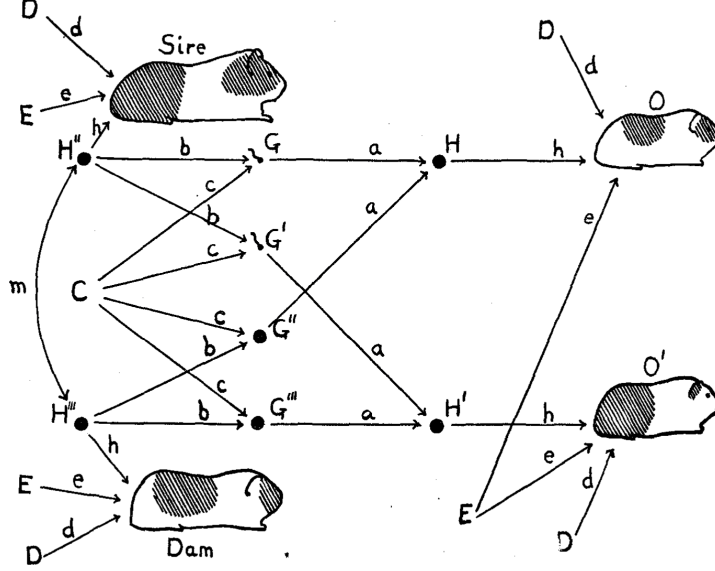


Figure 15: A path analysis diagram from [Wright \[1921\]](#). Wright used this diagram to visualize and analyze the relations between two mated individuals and their progeny.

Judea Pearl formalized Bayesian Networks (BN) in 1988 ([Pearl \[1988\]](#)). Similar to path analysis, BN uses nodes to represent variables and arrows (directed edges) to represent direct dependence. On top of that, BN accurately captures the conditional probability using the chain decomposition. Let $X_1, X_2, \dots, X_n$ be a collection of random variables and $p(x_1, \dots, x_n)$ their joint density function. We have

$$p(x_1, \dots, x_n) = p(x_1)p(x_2|x_1) \cdots p(x_n|x_1, \dots, x_{n-1}). \quad (3.2)$$

BN uses the decomposition, but based on the graph structure. So compared to path analysis, the computation of weight in BN produces actual probability. This greatly increase the information density of the same framework. Using BNs, we are able to effectively navigate through complex dependency relationships and compute the probability of events from evidence.

BNs have been widely used in various fields. Besides biology and medical research, BNs are also used in software engineering and finance. One of the most well-known application of BNs is the email spam filter ([Sahami et al. \[1998\]](#)). The early version of email spam filters had very simple structures. It looked at the overall probability of an email being potential spam and conditional probability when certain words are present. Intuitively it worked a lot like how we detect scam calls in some way. When we pick up a phone call, if the number is unknown, then we will likely raise our suspicion (conditional scam probability goes up). Then if the caller starts to ask for personal information or request money, then the conditional probability of the call being a scam goes up even more. We then most likely hang up because there is a Bayesian network-like process that just happens in our head. Today, email spam filters have got much more sophisticated, with some including LLMs. However, LLMs are expensive to run so traditional, BN-based email filters are still the mainstream choices, keeping billions of people safe from spam emails every day. For finance, the nature of BNs makes it the ideal tool to model potential risks ([Shenoy and Shenoy \[2003\]](#), [Gemela \[2001\]](#)). Risks and risk assessment can be generalized as attempts to answer the “what if” questions. In our daily lives, such questions can be easy. “What if I got a flat tire?” “Prepare a tire repair kit in your trunk.” Financial market and related risks have much more intertwined underlying causes. Such needs directly overlap with the specialty of BNs, making it the ideal tool for risk assessment and management til this day.

Now let us turn our attention to Markov Random Field (MRF). While Bayesian networks are very effective in representing direct dependencies and causal relationships. There are other types of links lacking clear “directionality”. As with the pixels of a picture we previously mentioned. It is clear that two adjacent pixels in a picture are likely to be correlated because they can depict the same object. However, it is impossible, or more accurately, logically unsound to say that one pixel causes the other. There are a lot more examples where the correlations are undirected or symmetric. In order to model those probabilistic dependencies, we need a different PGM other than BN. That is where MRF comes in.

The formalization of MRF is widely considered to be the work of Besag in 1974 (Besag [1974]). In this famous paper, Besag also presented an alternate proof for the famous Hammersley-Clifford theorem, showcasing the MRF as a powerful tool in terms of modeling dependencies based on geographic proximity. We will not go into the details of the proof, but it would be beneficial to state the theorem and compare the notion of MRF with a traditional Markov Chain.

Let $[n] = \{1, 2, \dots, n\}$ and $\mathcal{X} = (X_t)_{t \in [n]}$ be a finite dimension random vector with each X_t taking values in a finite set $\mathcal{S}_t = \{1, 2, \dots, m_t\}$. Suppose $[n]$ is the set of vertices of a graph. For each t , denote $\mathcal{N}_t$ the set of vertices s such that s is adjacent to t in the graph.

Definition 2. *The process $(X_t)_{t \in [n]}$ is said to be a Markov random field if*

(i) *for each tuple $x = (x_1, x_2, \dots, x_n) \in \prod_{i=1}^n \mathcal{S}_i$, the joint probability satisfies*

$$P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) > 0; \quad (3.3)$$

(ii) *for each t and x ,*

$$\mathbb{P}\{X_t = x_t | X_s = x_s, s \neq t\} = \mathbb{P}\{X_t = x_t | X_s = x_s, s \in \mathcal{N}_t\}. \quad (3.4)$$

We can see that the definition of MRF resembles much of the discrete-time Markov chain. Instead of the sequential Markov property, MRF has its own Markov property induced by the graph topology. A subset of vertices $A \subset [n]$ is said to be complete (or a clique) if each pair of vertices is connected with an edge in the graph. We can use this notion to define a Gibbs distribution.

Definition 3. *Let $x = (x_1, x_2, \dots, x_n) \in \prod_{i=1}^n \mathcal{S}_i$ be an arbitrary tuple from the joint distribution, and $\mathcal{C}$ the collection of all complete vertex subsets. A probability distribution Q is called a Gibbs distribution (or Gibbs random field) if it can be written in the form*

$$Q(x) = \prod_{A \in \mathcal{C}} V_A(x), \quad (3.5)$$

where $V_A(x)$ is a positive function that depends only on x_i 's with $i \in A$.

It is clear that each process $(X_t)_{t \in [n]}$ has a corresponding Gibbs distribution based on the dependencies. Now we can state the powerful Hammersley-Clifford theorem:

Theorem 3 (Hammersley-Clifford theorem). *Let Q be a probability distribution on $\prod_{i=1}^n \mathcal{S}_i$. Then Q is a Gibbs distribution if and only if it is a Markov random field with respect to the same graph structure.*

Besag's paper is a major cornerstone in statistics and fields dealing with spatial data. Before his work, defining a valid joint probability distribution over many interacting variables was often difficult. Besag showed how one could specify the model based on conditional probability of each variable given its neighbors (using the local Markov property). Such practice is much more intuitive and actionable. On the other hand, his paper did not stop

at a theoretical level. Besag also proposed practical methods for statistical analysis using pseudo-likelihood, which uses conditional probabilities to approximate maximum likelihood. Such estimations used to be intractable due to computational difficulties. Bridging theory and application, this paper was a huge achievement in the field of statistical learning that inspired countless advancements after it, such as the work by Geman and Geman (Geman and Geman [1984]) we mentioned in section 3.1.

Image Restoration

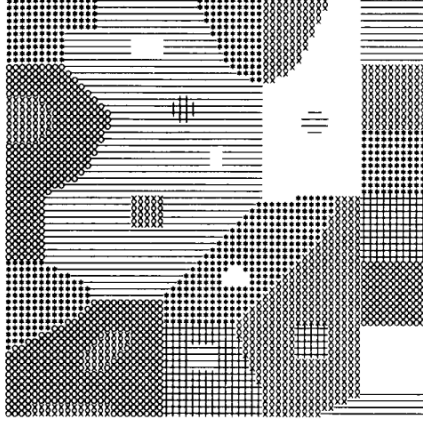


Fig. 3a. True six-colour scene: 64×64 .

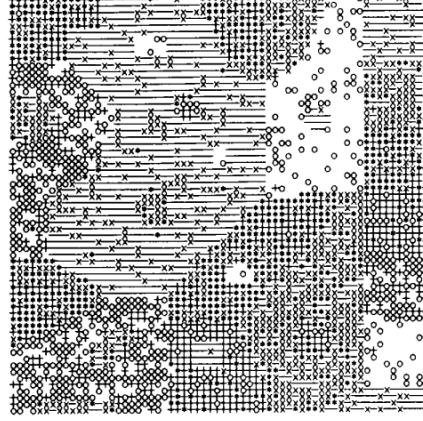


Fig. 3b. Maximum likelihood classifier: 32% error rate.

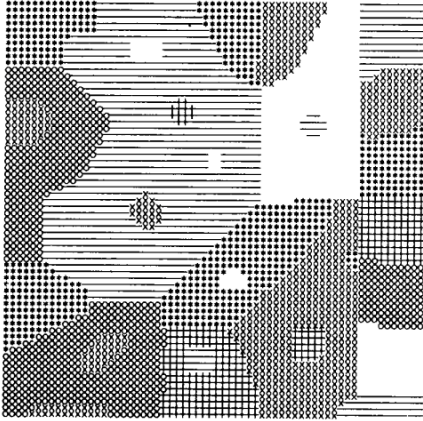


Fig. 3c. ICM reconstruction with $\beta \uparrow 1.5$: 1.2% error rate.

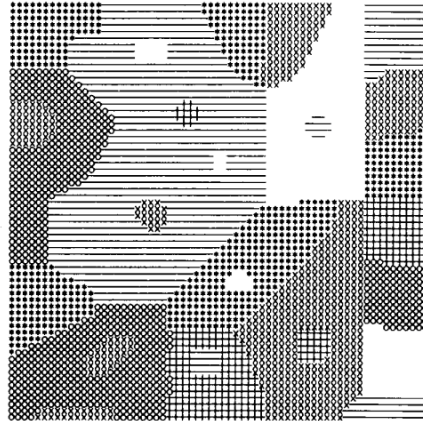


Fig. 3d. ICM reconstruction with β estimated: $\hat{\beta} = 1.80$: 1.1% error rate.

Figure 16: Besag used MRF to develop the Iterated Condition Modes (ICM) algorithm that is able to restore dirty images. The figure is from his work in 1986 Besag [1986]. As we can see, ICM performs much better than the old maximum-likelihood classifier.

The introduction of BN and MRF made researchers realize the power of PGMs. Also, graph theory and probability happen to be two of the mathematical sub-fields that are most related to computer science. People started to give PGMs much more focus, aiming to unify the models and theories so that a more comprehensive framework could serve to tackle the dependency problems emerging from complex systems and machine learning.

Factor graph was created for this very purpose in 2001 (Kschischang et al. [2001]). As the name suggests, factor graphs provide a general representation that explicitly shows the factorization of a global function into local factors. The joint probability we discussed above for MRF is a good example. They connect widely used models like MRF and BN, then unify the inference algorithms like belief propagation. The lineage from both MRF and BN

is very clear.

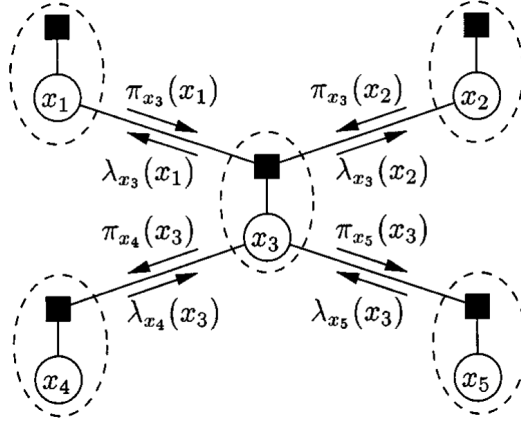


Figure 17: The diagram illustrating belief propagation in [Kschischang et al. \[2001\]](#). Belief propagation is a powerful model built upon BN and MRF that are broadly used in information theory and other related fields.

There were also other new ideas rising from the fertile soil of PGMs, such as Junction Trees, Variational Inference and MCMC ([Jordan et al. \[1999\]](#), [Gelfand and \[1990\]](#)). On the larger scale, the development of machine learning was also gradually shifting the fashion of the field. Instead of relying on expert knowledge, people began to use ML algorithms to learn the unknown parameters or the graph structures from complex data, building the navigation chart amidst the black sea of chaos. The effectiveness of PGMs, combined with the strong generative power of ML, expand the potentials of PGMs even further. It is exciting to see what this field will bring us for the rest of this century.

3.3 Bayesian Networks and Markov Random Fields

As we mentioned in section 3.2, Bayesian networks and Markov random fields represent the two main types of PGMs, reflecting to choice of directed or undirected graph. In chapter 3, we will provide a self-contained walkthrough on how machine learning algorithms can be used to learn the graph structure. Towards that end, in this chapter, we will dedicate some space to introduce the two main PGMs themselves, as well as the Lovász Local Lemma. This way we will have a better understanding of the background and how chapter 3 weaves different topics together to create a new algorithm.

Let us start with Bayesian network. Let $p(x_1, \dots, x_n)$ be any probability density function. Recall that we can decompose p using a chain of condition probability:

$$p(x_1, \dots, x_n) = p(x_1)p(x_2|x_1) \cdots p(x_n|x_1, \dots, x_{n-1}). \quad (3.6)$$

Such decomposition is often computationally intractable. For BN, we modify such decomposition that each x_i only depends on a small subset of ancestor variables x_{A_i} . For example, whether one takes the walk or bus depends on tomorrow's weather, and the weather depends a series of complicated natural factors. It would be an overkill if we model the choices of transportation on all these factors. Instead, we just let the weather be the only ancestor variable of the transportation choices and weather itself depends on the natural factors such as temperature, wind direction, etc. Bearing the intuition in mind, we can now give the formal definition of BN.

Definition 4. A Bayesian network is a directed acyclic graph (DAG) $G = (V := [n], E)$ such that:

- (i) each node i represents a random variable X_i ;
- (ii) for each X_i , there is a corresponding conditional probability distribution $p(x_i|x_{A_i})$ where A_i consists of all the immediate ancestors of i in the graph G .

Conversely, we say a probability distribution p factorizes over a DAG G if it can be decomposed into a product of factors induced by G .

Let us take a look at a simple example from [Koller and Friedman \[2009\]](#).

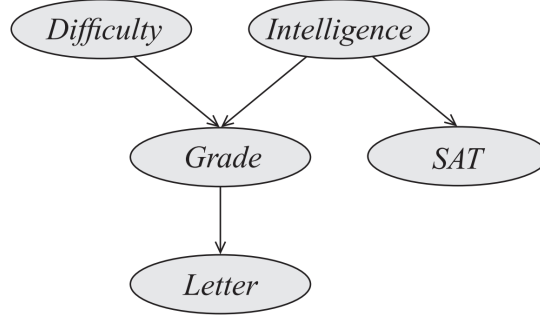


Figure 18: Figure 3.3 from [Koller and Friedman \[2009\]](#), a simple BN that models the letter graph and SAT scores of students.

Denote difficulties, grade, intelligence, SAT, and Letter by their first letter d, i, g, s, l respectively. We can decompose the joint probability $p(d, i, g, s, l)$ in figure 18 starting from end nodes s and l and moving upwards through the ancestry:

$$p(d, i, g, s, l) = p(s|i) \cdot p(l|g)p(g|i, d)p(i)p(d). \quad (3.7)$$

The reason that the example is well constructed is that not only it provides an easy-to-follow computation example, but also provides the three main patterns of conditional dependence in BN, based on whether the ancestor or intermediate node is observed or not. For three random variables X, Y, Z , we write $X \perp Y | Z$ if X and Y are independent condition on Z . We also use arrows to denote the local graph structure. For example, $X \rightarrow Y$ means there is a directed edge from X to Y . The three common patterns are as follows.

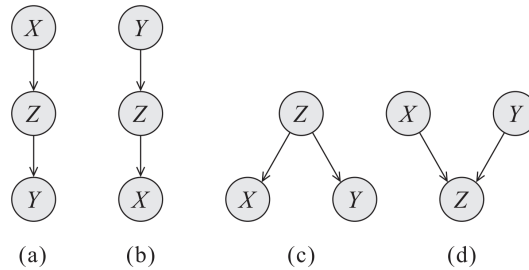


Figure 19: Figure 3.5 from [Koller and Friedman \[2009\]](#) that shows common connecting patterns in BN.

- (i) Common cause, corresponding to (c) in the figure 19 above. If Z is unobserved, then obviously X and Y are not independent of each other because they share a common part

of their randomness. However, if we know Z , then $X \perp Y | Z$ as the shared common part is determined and the rest of randomness is independent. To see that mathematically, first notice that we can write the joint probability as

$$p(X, Y, Z) = p(X|Z)p(Y|Z)p(Z). \quad (3.8)$$

For any x, y, z , z fixed, we have

$$\begin{aligned} \mathbb{P}(X = x, Y = y | Z = z) &= \mathbb{P}(X = x | Z = z) \mathbb{P}(Y = y | Z = z) \mathbb{P}(Z = z | Z = z) \\ &= \mathbb{P}(X = x | Z = z) \mathbb{P}(Y = y | Z = z) \end{aligned} \quad (3.9)$$

by the definition of conditional independence.

(ii) Common effect, which is (d) in the figure 19. If Z is unobserved, then X and Y are independent of each other. However, if we know Z , then X and Y are not independent of each other. The mathematical proof follows similarly as the example in (i) above. Moreover, it makes intuitive sense if we look at figure 18 for the relationship among difficulty, intelligence and grade. Of course, the difficulty of the exams and the intelligence of students are independent as they are the results from different entities (exam makers and students). However, if we know the grade of a student, say a very high grade for a hard exam, that will change the joint probability. Performing well on a hard exam is more likely to come from a smart student. Difficulty and intelligence are no longer independent in this case.

(iii) Evidential and causal effects, representing (a) and (b) in figure 19. Clearly for unobserved Z we have X and Y are not independent. However, if the intermediate node Z is observed, then $X \perp Y | Z$. This means that everything it takes to determine the final state is provided, so the root in the network no longer plays a role in the outcome of its children. This is also clear in figure 18. If the exam result is released, then the letter grade of a student will depend solely on the score, not on the smartness of the student.

The three patterns can be used to define another important concept called d-separation. As we just see, dependency can be passed through paths on the graph via a series of 3-node patterns given observed data. If there are no such way to connect two clusters of nodes, then they are essentially conditionally independent, revealing insights of the data we are trying to analyze.

Definition 5 (d-separation). *Let $\mathcal{X}$, $\mathcal{Y}$ and $\mathcal{O}$ be three sets of nodes in a Bayesian network G . We say that $\mathcal{X}$ and $\mathcal{Y}$ are d-separated given $\mathcal{O}$ (The letter $\mathcal{O}$ stands for observed) if $\mathcal{X}$ and $\mathcal{Y}$ are not connected by an active path. An undirected path in G (meaning one can go against the arrows) is called active given observed nodes $\mathcal{O}$ if for every sequence of consecutive three nodes X, Y, Z on the path satisfies one of the following conditions:*

- (i) $X \leftarrow Y \leftarrow Z$, $Y \notin \mathcal{O}$;
- (ii) $X \rightarrow Y \rightarrow Z$, $Y \notin \mathcal{O}$;
- (iii) $X \rightarrow Y \leftarrow Z$, $Y \in \mathcal{O}$.
- (iv) $X \leftarrow Y \rightarrow Z$, $Y \notin \mathcal{O}$.

In figure 20, we mark the observed nodes with a darker gray color. It is easy to check that node A and F are d-separated given B and C on the left, while B and C are not d-separated given A and F on the right ($B \rightarrow D \rightarrow F \rightarrow E \rightarrow C$ is an active path). We then proceed to define

$$\mathcal{I}(G) = \{(\mathcal{X}, \mathcal{Y}, \mathcal{Z}) | \mathcal{X}, \mathcal{Y} \text{ are d-separated by } \mathcal{Z}\}. \quad (3.10)$$

This definition is where the power of graph theory starts to manifest. Recall that graph theory studies quantities about graphs that remain unchanged under various transformation

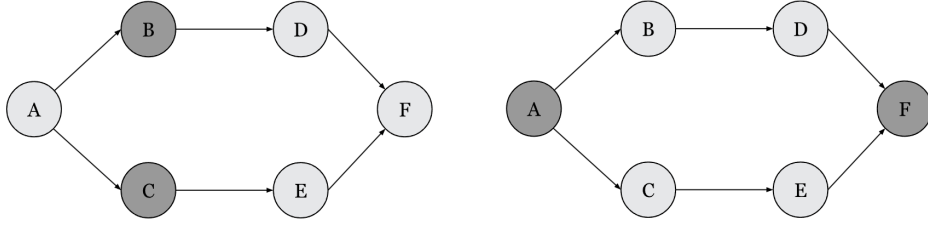


Figure 20: A pair BNs illustrating the concept of d-separation.

and mapping, such as coloring and characteristic polynomial. The quantity $\mathcal{I}(G)$ is also one of such quantities tied to Bayesian network that helps identify different Bayesian networks with similar underlying structures. If we define two BNs G and G' as I-equivalent if $\mathcal{I}(G) = \mathcal{I}(G')$, then the following theorem holds

Theorem 4 (Theorem 3.9, [Koller and Friedman \[2009\]](#)). *Two Bayesian networks G and G' are I-equivalent if and only if there exists a sequence of network*

$$G = G_1, \dots, G_k = G' \quad (3.11)$$

such that the only difference between G_i and G_{i+1} , $1 \leq i \leq k-1$ is a single reversal of a covered edge.

This theorem, combined with an algorithm that can find $\mathcal{I}(G)$ and machine learning algorithms that can learn the graph structure, provides a capable framework for studying highly complex objects with lots of intricate dependencies. There are many other fascinating facts about BN but we should stop here to prevent ourselves from deviating too far. Now we will turn our attention to MRF.

Unlike BNs, the definition of an MRF does not derive strictly from decomposition of conditional probability. Rather, it assigns a factor(weight) to each clique(complete subgraph) in the graph.

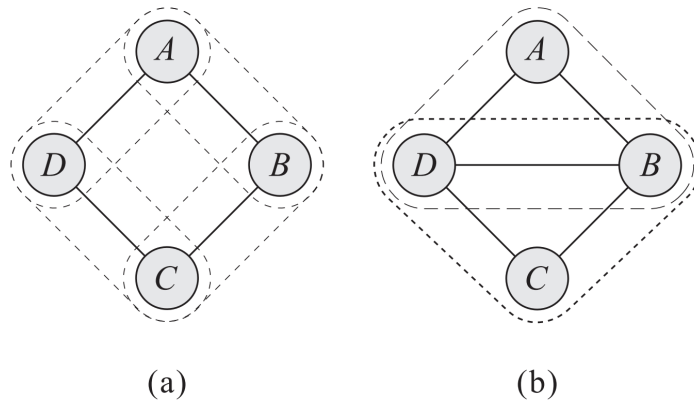


Figure 21: Two MRFs with different edges (and thus cliques) from figure 4.4, [Koller and Friedman \[2009\]](#).

Take figure 21 for instance, in the left graph, there are four complete subgraphs (A, B) , (B, C) , (C, D) and (A, D) . For any value assignments (a, b, c, d) of the random variables (A, B, C, D) , we can write the factor of the value assignment as

$$\tilde{p}(a, b, c, d) = \phi(a, b)\phi(b, c)\phi(c, d)\phi(a, d), \quad (3.12)$$

where ϕ can be any positive functions defined by the user. Normalizing the factor will give us the joint probability distribution of the MRF:

$$p(a, b, c, d) = \frac{1}{Z} \tilde{p}(a, b, c, d), \quad (3.13)$$

and

$$Z = \sum_{(a, b, c, d)} \tilde{p}(a, b, c, d) \quad (3.14)$$

is called the partition function. We have already defined MRF in definition 2. For the graph on the right, there are only two complete subgraphs, consisting of A, B, D and B, C, D . Then the factor score has the form

$$\tilde{p}(a, b, c, d) = \phi(a, b, d)\phi(b, c, d). \quad (3.15)$$

Now we can give an alternative and equivalent definition based on the graph structure:

Definition 6 (Alternative Definition of MRF). *Let $X_1, \dots, X_n$ be random variables. A Markov Random Field (MRF) is the joint probability distribution of $(X_i)_{i \in [n]}$ over an undirected graph $G = (V := [n], E)$ with each vertex i representing the random variable X_i . In addition, the joint probability p has the form*

$$p(x_1, \dots, x_n) = \frac{1}{Z} \prod_{C \in \mathcal{C}} \phi_C(x_C), \quad (3.16)$$

where $\mathcal{C}$ is the collection of all cliques (complete subgraphs) in G , each factor ϕ_C is a non-negative function, and

$$Z = \sum_{x_1, \dots, x_n} \prod_{C \in \mathcal{C}} \phi_C(x_C) \quad (3.17)$$

is the normalizing partition function.

While both are powerful, MRFs and BNs excel at modeling different types of dependencies. Although neither is strictly more general, any BN distribution can be represented by an MRF through a process called moralization. This involves adding an edge between any pair of parents that share a common child ('marrying parents') and then making all edges undirected. Furthermore, the concept of conditional independence is defined differently: while BNs use d-separation via active paths, two sets of nodes in an MRF are conditionally independent given a third set if that third set separates them in the graph. The idea follows similarly as the evidential and causal effects in BN: a path of nodes connecting two clusters is the shared part of their randomness. If the path is fully observed, then the rest of randomness is conditionally independent. In particular, if all neighbors of a node are observed, then this node is independent from the rest of the network.

Upon this point, we can see that MRF has several advantages over BN. First, MRF is more versatile. We can apply MRF to a broader range of problems. Any BNs can be converted to MRF but not vice versa, and especially when the dependencies of the model are naturally symmetric. This gives us the second advantage, that MRFs are able to describe certain dependency structures that cannot be captured by BNs like k -CNF formulas.

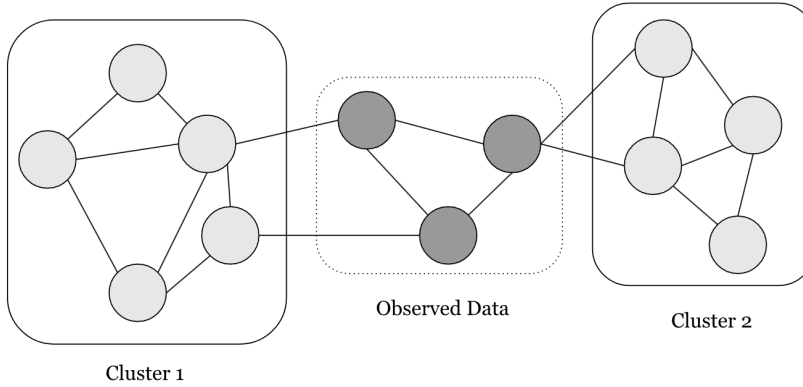


Figure 22: An MRF with two indicated independent clusters. The nodes with darker grey in the middle are observed, causing the clusters on the left and right to be independent.

However, there are reasons that we cannot just use MRF everywhere. The primary drawback of MRF is the computation of Z . To compute the partition function Z , one might need to sum over an exponential number of possible value assignments. This summation is generally NP-hard. Moreover, since every BN is required to be acyclic and directed, it is very easy to generate data using the hierarchy of dependencies relations. Since MRF, in contrast, is undirected and thus has no “starting points” or ancestors, it is often much harder to generate data from a MRF.

Since our work in chapter 3 mainly deals with intractable computations, we will dedicate some space to discuss sampling and Lovász Local Lemma in the next section. We also encourage the reader to check out the textbook by Koller and Friedman [Koller and Friedman \[2009\]](#) for a more comprehensive introduction to PGMs, which is considered the standard textbook on the subject.

3.4 The Necessity of Sampling and Lovász Local Lemma

In the previous section, we mentioned that computing the partition function Z in an MRF directly is likely to be computationally intractable. Therefore, sampling algorithms are usually required so that we can avoid computations that scale exponentially. This is a clever and natural idea and approach, but it introduces a new problem.

As the theme throughout this section, dependency is everywhere in the data we are trying to analyze. It often comes in the form of constraints, or a series of “bad events”. For the machine learning algorithm to effectively learn the PGM model, we need to make sure the samples we generate are from the actual conditional distributions from the data. Therefore when we are trying to sampling with constraints, it is important for us to know the (approximate) answers of the following two questions:

- (i) Given a set of constraints, do the samples satisfying all conditions exist?
- (ii) If yes, how do we sample from the distribution effectively without spending exponential time or stuck in an finite loop. (Meeting one constraint will fail another. Sometimes constant adjustment without proper strategy will indefinitely prolong the sampling process.)

Though there is no one-size-fits-all answer to the two questions, there are some general principles that can be applied to a wide range of problems. One of the most famous and

widely used principle is Lovász Local Lemma (LLL), discovered and proved by Erdős and Lovász in 1975.

Lemma 1 (Erdős and László [1974]). *Let $A_1, \dots, A_k$ be a sequence of events such that each event occurs with probability at most p and such that each event is independent of all the other events except for at most d of them. Assume*

$$4pd \leq 1. \quad (3.18)$$

Then there is a nonzero probability that none of the events occurs.

Shearer improved the results in 1985 and the updated lemma becomes the Lovász Local Lemma (LLL) we know today:

Lemma 2 (Shearer [1985]). *Assuming the same conditions as lemma 1, if we have $p \leq \frac{1}{2d}$. Assume*

$$\begin{aligned} p &< \frac{(d-1)^{(d-1)}}{d^d}, \quad d > 1 \\ p &< \frac{1}{2}, \quad d = 1. \end{aligned} \quad (3.19)$$

Then there is a nonzero probability that none of the events occurs.

Both lemma 1 and 2 answered the first existence problem. However, there is a lot of room for improvement. All the bad events have a common upper bound p , which reduces the applicability of the lemmas. Spencer in 1977 proved the asymmetric version of the lemma. To state this result, we need to the concept of dependency graphs, which are extensively used in chapter 3. To put in simple terms, a dependency graph is a graph with nodes representing a set of events such that mutually independent events are not adjacent, otherwise connected via an edge. With that being said, we have the following statement.

Lemma 3 (Spencer [1977], Asymmetric Lovász local lemma). *Let $\mathcal{A} = \{A_1, \dots, A_n\}$ be a finite set of events in the probability space Ω . For $A \in \mathcal{A}$, let $\Gamma(A)$ be the neighbors of A in the dependency graph. Assume there exists a map $f : \mathcal{A} \rightarrow [0, 1]$ such that for every $A \in \mathcal{A}$,*

$$\mathbb{P}(A) \leq f(A) \prod_{B \in \Gamma(A)} (1 - f(B)). \quad (3.20)$$

Then there exists a nonzero probability that none of the events in $\mathcal{A}$ occurs. Additionally

$$\mathbb{P}\left(\bigcap_{i=1}^n \bar{A}_i\right) \geq \prod_{A \in \mathcal{A}} (1 - f(A)). \quad (3.21)$$

As we can see, the asymmetric LLL has a stronger result, even with a positive lower bound. However, the improved existence does not lead to a actionable algorithm that helps us sample from the distribution. All proofs of the three lemmas above are non-constructive. In order to put this powerful mathematical fact into practice, there has been many attempts seeking for constructive proofs or better, an algorithmic version of LLL. To name a few, Beck in 1991 gave an algorithmic proof of LLL under polynomial time, one of the early milestones that turned LLL into an effective sampling technique. The most well-known result is probably by Moser and Tardos (Moser and Tardos [2009]). Because of their improvement and novelty, Robin Moser and Gábor Tardos received the Gödel Prize in 2020.

Thus far, we have introduced most background knowledge related to our work in chapter 3. We started with the importance of the overarching dependency problem, then proceeded to give a brief history of the tools we use. Finally, we introduced LLL, which we later incorporate into a neural solver with parallelism capability. Starting from graph theory and probability, the intersection between various fields of mathematics and computer science never ceases to amaze us with its beauty and potential.

4 Factor Analysis and Variational Inference

4.1 Combining DR and PGM

In previous sections 2 and 3, we have discussed the importance and difficulties of problems in probability and data science that arise from high dimensionality and dependencies. To solve these two problems, scientists created dimension reduction and probabilistic graphic models to tackle them respectively. For our work in chapter 4, the algorithm we develop involves both dimension reduction and probabilistic graphic models. The framework we invented is called Variational inference Factor Analysis (VarFA). It is self-explanatory that this toolkit combines variational inference with factor analysis.

From the dimension reduction perspective, the variational inference part draws inspiration of classical Variational AutoEncoder (VAE), which utilizes the so-called “latent dimensions” to generate output similar to the input data set. To understand latent dimensions, let us take a look at some real-life examples, such as modeling in games. For large games (such as games with vast open worlds) it is impossible to create models for every single NPC, building and tree, etc. One could use the same object repetitively, but seeing two identical people on the street will definitely break immersion. Instead, game developers want to generate a set of objects that are similar to the in-game object they have already created, but different. If we see input image X as high-dimensional data with pixel information, then we want to find a probabilistic algorithm that can compute the probability $p(X)$, where p refers to the probability of how likely this collection of pixel can be seen as similar objects created manually by developers. If p is high, we take the image and a new in-game object is generated.

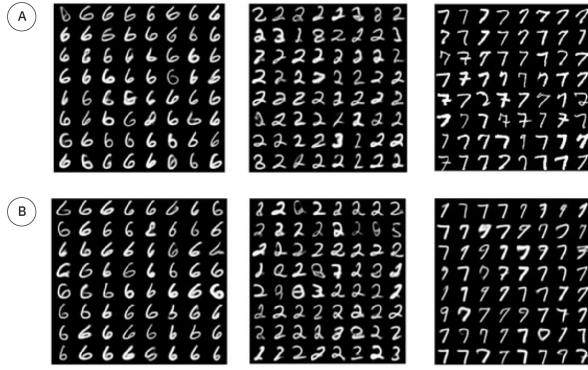


Figure 23: The original handwritten digits from MNIST (top row) and VAE-generated digits (bottom row), from [Mak et al. \[2023\]](#). Though some outputs are not entirely natural, all are recognizable and there is no mixing of two different digits.

But the task is not as easy as it seems. If we use brutal force approaches, we can end up with an NPC with body of a human and head of a bus, if the pool of hand-drawn objects contain both humans and buses. This will create a lot of trouble. However, if the model is designed to assign a category to each of the objects, such as human, vehicles, plants, etc, then it will know not to mix things up when generating objects. The vector (human, vehicles, plants, ...) is the latent dimension we are talking about. In practice, latent dimensions usually do not have meaningful labels, and they do not need to. Just as t-SNE, the latent dimensions are used to represent different types of data samples and make sure the output is consistent with the categorization/representing clusters.

As for PGM viewpoint, the architecture of VarFA can be represented as a neural network (directed graph), where each layers of input determines the values/distributions of the next layers. The training process of the model is essentially learning the parameters on the graph. This firmly places VarFA in the PGM family. Indeed, as we can see in chapter 4, we model the performance/grades of students based on latent skill levels. How well a student perform on a specific test question can be a combination of related skills. VarFA establishes the dependency structures between competence and exam scores, not just predicting them.

The only background we have not covered is factor analysis. In the next section, we will cover the basic concepts of factor analysis as well as its historical heritage. Afterwards, we will discuss the concepts and techniques we borrow from VAE. With all the topics covered, the readers should have a sufficient context to understand the motivation and contribution of our work in chapter 4.

4.2 Lineage of Factor Analysis

We have introduced the brief history of dimension reduction in section 2.2, including a brief reference to VAE. In fact, Factor Analysis (FA) is also a dimension reduction technique, but not in the same fashion as the other algorithms we introduced earlier. Therefore, it would be beneficial to give a short rundown of this approach before we move onto the next section.

FA was widely considered to be the brain child of Charles Spearman (Spearman [1904]), the famous statistician and psychologist known for the famous “Spearman Correlation”. In his famous paper, he conducted a series of experiments on the performance of students in different school subjects and found that although a lot of the subjects were seemingly unrelated, the students’ performance was consistent across the board. This led him to think that there was one single underlying factor that determines the competence of students, or colloquially speaking, “smart kids can learn any subject quickly and easily”. What he proposed was the so-called single-factor model, associating students’ performance with a single variable called “general mental ability”. Coincidentally, VarFA also focuses its application in education and the modelling of students’ performance. It is no doubt that education is more than vital to the entire human society, and that is precisely why we continue to go back to such topic, more so in the era of machine learning and AI.

After Spearman’s idea, various researchers tried to expand the model to naturally include the cases where multiple factors are required to model and predict the data. One of the most important cornerstones in this field is the work by Thurstone (Thurstone [1947]). This book includes various important works starting from the 1930s. The discoveries in this period laid the foundation of the FA framework we are familiar with today.

In particular, assume there are p rounds of observations for n individuals. FA assumes that the collective behavior of the individuals can be explained by k common factors, and $k < p$. Mathematically speaking,

$$x_{i,m} - \mu_i = \sum_{j=1}^k l_{i,j} f_{j,m} + \epsilon_{i,m}. \quad (4.1)$$

Here, $x_{i,m}$ is the i -th observation of the m -th individual, μ_i is the mean of the i -th observation, $l_{i,j}$ is the loading of the j -th factor on the i -th observation, $f_{j,m}$ is the j -th factor score for the m -th individual and $\epsilon_{i,m}$ is the error term, usually with zero mean and finite variance. This definition is the formulation of basically one sentence:

For the samples we are investigating, the observations are linear combinations of the common factors plus some noise.

And it is easy to see that (4.1) can be rewritten in the form of matrix multiplication:

$$X - M = LF + \epsilon. \quad (4.2)$$

More details about factor analysis will be given in chapter 4. In this introductory chapter, it is more important to address the unaddressed statement at the beginning of this section: why is factor analysis different from other dimension reduction techniques, though it seems very basic, only using linear combinations and independent noise? The answer lies not in the mathematical formulation, but rather the philosophy of usage. **FA is the quantitative catalyst of statistical intuition.**

To explain this answer, let us point out that FA can be divided into two categories: Exploratory Factor Analysis (EFA) and Confirmatory Factor Analysis (CFA). EFA is created in 1930s and largely contributed by Thurstone (Thurstone [1947]). In EFA, the user does not specify the structure of observations and is required to determine the number of factors. Afterwards, the user needs to choose a rotation method to make the factors more interpretable. Unlike PCA, EFA’s goal is to help researchers interpret the latent factors, whereas PCA stops after giving the data points in lower dimensions. EFA aims to generate intuitions. Though closely related to PCA, the mind set behind EFA was already different even in early 20th century.

This unique philosophy of EFA was extended further when CFA was created in 1960s by Jöreskog (Jöreskog [1967], Jöreskog [1969]). The user now needs to specify the structure of the model, from the number of factors and the loadings for each of them. CFA was created to provide a statistically sound procedure to verify what might be considered “a good guess from experienced researchers”. To use this model, researchers are required to think carefully about the underlying structure of their measurements, and provide a plausible model to evaluate those thoughts. CFA tests intuition.

The DR techniques we introduced in section 2 are mostly automatic, input the data and the algorithm will do the rest for you. Meanwhile, FA values human input and strives to make the results explainable. Though a lot of advancement in today’s AI comes with the caveat of “black box”, FA’s lineage was well preserved and inspired a lot of subsequent discoveries in the field of Explainable Artificial Intelligence (XAI) (Doshi-Velez and Kim [2017], Miller [2018]). It is also the reason why we choose FA to combine with the generative power of VAE. Our work in chapter 4 is not a mere mix and swap of models, but rather a concrete practice of new mindsets towards the goal of fostering human learning with machine learning.

4.3 The Modern Toolkit: Variational Autoencoders

Apart from FA, VAE is another core piece of the puzzle in our work about VarFA. We brought up VAE in section 2.2 briefly and now it is a good time to formally introduce it. Similar to algorithms like t-SNE, VAE uses the so-called latent dimensions that are often not interpretable. However, unlike t-SNE, VAE is a generative model, meaning instead of visualization, it is to generate new data points that follow the same distribution as the input data. Invented by Welling and Kingma in 2013 (Kingma and Welling [2014]), not long after CNN was coined. VAE is an important milestone in the field of artificial intelligence because the capability of generating content based on training materials back then was truly revolutionary.

That being said, VAE is not the silver bullet, far from it. As we will see very soon in the deduction of the method, VAE relies on a lot of bold assumptions and approximations, and offers little room for tuning. Unlike t-SNE and MRF, VAE is not used in chapter 4, but extremely important in the overall literature. Therefore we will give a more detailed walkthrough of this amazing algorithm, focusing on the underlying assumptions and more

explicit mathematical details. (There are many other more standard materials on VAE, such as the famous tutorial by Carl Doersch [Doersch \[2021\]](#).)

Given a data point $\mathbf{x}$ in high dimension, VAE assumes that $\mathbf{x}$ is determined by a fewer number of latent variables, denoted as a lower dimensional vector $\mathbf{z}$. Using conditional probability decomposition, we can write

$$p(\mathbf{x}|\mathbf{z}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z} \equiv \int p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}, \quad (4.3)$$

where θ is the hyperparameter VAE is trying to learn. The exact formulation is given in (4.19) and (4.23). The true bold claim here is not that data itself depends on latent variables, but the assumption that $p(\mathbf{z})$ is (often) a multi-dimensional random variable with i.i.d., standard normal entries. Bold as it might be, the assumption itself is in fact pretty well-grounded. For instance, your performance at a certain sport is determined by a series of factors such as your endurance and strength. It would be normal that both your strength and endurance follow a Gaussian-like distribution. Your endurance might fluctuate around your “average” level and performing significantly above or below your average is getting exponentially rare. Since all Gaussian distributions are the result of a simple affine transformation of the standard normal distribution, we might as well assume that the latent variables are i.i.d. standard normal from the very beginning. Furthermore, p_θ is also mostly set to be a Gaussian distribution if the target data is continuous following the same logic.

After giving $p(\mathbf{z})$ a distribution, determining the parameters for $p_\theta(\mathbf{x}|\mathbf{z})$ directly is not feasible. That is the primal motivation for the encoder-decoder architecture. By Bayes formula, we can write

$$p_\theta(\mathbf{z}|\mathbf{x}) = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})} = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{\int p_\theta(\mathbf{x}|\mathbf{z}')p(\mathbf{z}')d\mathbf{z}'}. \quad (4.4)$$

This means we want to learn the relationship between the input data and the latent factors first before we start generating data. This way we can use the learned hyperparameter θ and samples from $p(\mathbf{z})$ to generate new data points. This leads to the next problem: $p(\mathbf{x})$ is also intractable. Therefore, we need another parametrized Gaussian distribution $q_\phi(\mathbf{z}|\mathbf{x})$ to approximate $p_\theta(\mathbf{z}|\mathbf{x})$. This is the second bold assumption, that the conditional distribution of latent variables given observed data is (almost) Gaussian. To give an intuitive example, suppose we see a student performing exceptionally well in a math exam, we might take a high guess at the student’s math skill (mean), or the related latent variable. If the student is consistent, then we might decrease the variance. Formally speaking, $q_\phi(\mathbf{z}|\mathbf{x})$ takes the following form:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))). \quad (4.5)$$

Here, $\boldsymbol{\mu}_\phi(\mathbf{x})$ is the mean vector. $\boldsymbol{\sigma}_\phi^2(\mathbf{x})$ is the variance vector (diagonal elements of the covariance matrix). The encoder network takes $\mathbf{x}$ as input and outputs the parameters $\boldsymbol{\mu}_\phi(\mathbf{x})$ and $\boldsymbol{\sigma}_\phi^2(\mathbf{x})$ (or often $\log \boldsymbol{\sigma}_\phi^2(\mathbf{x})$ for numerical stability). The explicit formula for the probability density function (PDF) of $q_\phi(\mathbf{z}|\mathbf{x})$, assuming a N -dimensional latent space, is:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \frac{1}{(2\pi)^{N/2}|\boldsymbol{\Sigma}_\phi(\mathbf{x})|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{z} - \boldsymbol{\mu}_\phi(\mathbf{x}))^T \boldsymbol{\Sigma}_\phi(\mathbf{x})^{-1}(\mathbf{z} - \boldsymbol{\mu}_\phi(\mathbf{x}))\right) \quad (4.6)$$

where $|\boldsymbol{\Sigma}_\phi(\mathbf{x})|$ is the determinant of the covariance matrix $\boldsymbol{\Sigma}_\phi(\mathbf{x})$, which simplifies to $\prod_{j=1}^N \sigma_{j,\phi}^2(\mathbf{x})$.

Let us take a pause here and recall that we make two bold assumptions during the setup of the encoder-decoder framework. We assume that the latent variables are i.i.d. standard normal random variables, and the conditional distribution of latent variables given observed

data is also Gaussian. Without these two assumptions, VAE will not be computationally possible. So qualitatively speaking, we can say VAE roots itself in one core belief:

The data we are working on is intrinsically Gaussian.

Now let us go back to the mathematical details. We call $p_\theta(\mathbf{z}|\mathbf{x})$ the true posterior distribution, the distribution “post the observation” and $p(\mathbf{z})$ the prior, the distribution “prior to the observation”. We measure the distance between our approximation $q_\phi(\mathbf{z}|\mathbf{x})$ and $p_\theta(\mathbf{z}|\mathbf{x})$ using KL-divergence. By definition 1, we compute

$$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) = \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z}|\mathbf{x})} d\mathbf{z} \quad (4.7)$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p_\theta(\mathbf{z}|\mathbf{x})] \quad (4.8)$$

Substitute $p_\theta(\mathbf{z}|\mathbf{x}) = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})}$:

$$\begin{aligned} D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) \\ = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p_\theta(\mathbf{x}|\mathbf{z}) - \log p(\mathbf{z}) + \log p(\mathbf{x})] \end{aligned} \quad (4.9)$$

Since $\log p(\mathbf{x})$ does not depend on $\mathbf{z}$, the expectation is just $\log p(\mathbf{x})$:

$$\begin{aligned} D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) \\ = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z})] - \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] + \log p(\mathbf{x}) \end{aligned} \quad (4.10)$$

The first term is the KL divergence between the approximate posterior and the prior:

$$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z})] \quad (4.11)$$

Substituting this back into the equation of $D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x}))$ gives:

$$\begin{aligned} D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) \\ = D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) - \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] + \log p(\mathbf{x}) \end{aligned} \quad (4.12)$$

Rearranging for $\log p(\mathbf{x})$:

$$\begin{aligned} \log p(\mathbf{x}) \\ = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) + D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) \end{aligned} \quad (4.13)$$

Since KL divergence is always non-negative ($D_{\text{KL}}(q || p) \geq 0$), we have:

$$\log p(\mathbf{x}) \geq \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) \quad (4.14)$$

This right-hand side is the Evidence Lower Bound (ELBO), often denoted $\mathcal{L}(\theta, \phi; \mathbf{x})$:

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \underbrace{\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction Term}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))}_{\text{KL Divergence / Regularization Term}} \quad (4.15)$$

Maximizing the ELBO with respect to parameters θ and ϕ serves two purposes:

1. It pushes $\log p(\mathbf{x})$ higher (our original goal).
2. It minimizes the KL divergence between the approximate posterior $q_\phi(\mathbf{z}|\mathbf{x})$ and the true posterior $p_\theta(\mathbf{z}|\mathbf{x})$ (because $\log p(\mathbf{x})$ is fixed for a given $\mathbf{x}$, and the ELBO is $\log p(\mathbf{x}) - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x}))$).

To making things mathematically explicit, let us take a look at the formula of ELBO (4.15). The KL divergence term measures how much the approximate posterior (learned by the encoder) deviates from the prior $p(\mathbf{z})$. Specifically, when

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))), \quad p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}) \quad (4.16)$$

this KL divergence has a convenient analytical solution:

$$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) = \frac{1}{2} \sum_{j=1}^N (\mu_{j,\phi}^2(\mathbf{x}) + \sigma_{j,\phi}^2(\mathbf{x}) - \log(\sigma_{j,\phi}^2(\mathbf{x})) - 1) \quad (4.17)$$

To see this, the general KL divergence between $Q = \mathcal{N}(\boldsymbol{\mu}_Q, \boldsymbol{\Sigma}_Q)$ and $P = \mathcal{N}(\boldsymbol{\mu}_P, \boldsymbol{\Sigma}_P)$ is

$$\frac{1}{2} \left(\text{Tr}(\boldsymbol{\Sigma}_P^{-1} \boldsymbol{\Sigma}_Q) + (\boldsymbol{\mu}_P - \boldsymbol{\mu}_Q)^T \boldsymbol{\Sigma}_P^{-1} (\boldsymbol{\mu}_P - \boldsymbol{\mu}_Q) - N + \log \frac{|\boldsymbol{\Sigma}_P|}{|\boldsymbol{\Sigma}_Q|} \right). \quad (4.18)$$

Plugging in $\boldsymbol{\mu}_Q = \boldsymbol{\mu}_\phi(\mathbf{x})$, $\boldsymbol{\Sigma}_Q = \text{diag}(\sigma_\phi^2(\mathbf{x}))$, $\boldsymbol{\mu}_P = \mathbf{0}$, and $\boldsymbol{\Sigma}_P = \mathbf{I}$ leads to the simplified formula above.

On the other hand, the reconstruction term usually depends on whether the data is binary or continuous. For binary data such as MNIST digits, we assume each pixel x_i is a Bernoulli random variable. The decoder network (parameterized by θ) takes $\mathbf{z}$ and outputs a vector $\mathbf{p}_\theta(\mathbf{z})$, where each element $p_{i,\theta}(\mathbf{z})$ is the probability that pixel x_i is 1.

$$p_\theta(\mathbf{x}|\mathbf{z}) = \prod_{i=1}^N p_{i,\theta}(\mathbf{z})^{x_i} (1 - p_{i,\theta}(\mathbf{z}))^{1-x_i} \quad (4.19)$$

where N is the dimensionality of $\mathbf{x}$. The log-likelihood is:

$$\log p_\theta(\mathbf{x}|\mathbf{z}) = \sum_{i=1}^N [x_i \log p_{i,\theta}(\mathbf{z}) + (1 - x_i) \log(1 - p_{i,\theta}(\mathbf{z}))] \quad (4.20)$$

This is the negative binary cross-entropy between the input $\mathbf{x}$ and the reconstructed probabilities $\mathbf{p}_\theta(\mathbf{z})$. The reconstruction term becomes:

$$\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\sum_{i=1}^N (x_i \log p_{i,\theta}(\mathbf{z}) + (1 - x_i) \log(1 - p_{i,\theta}(\mathbf{z}))) \right] \quad (4.21)$$

For continuous data, we assume each dimension x_i is Gaussian. The decoder network takes $\mathbf{z}$ and outputs the mean of the Gaussian $\boldsymbol{\mu}_\theta(\mathbf{z})$. We often assume a fixed variance, say $\sigma^2 \mathbf{I}$, for simplicity (e.g., $\sigma^2 = 1$), or the decoder could also output the variance. Assuming fixed variance σ^2 :

$$p_\theta(\mathbf{x}|\mathbf{z}) = \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x_i - \mu_{i,\theta}(\mathbf{z}))^2}{2\sigma^2} \right) \quad (4.22)$$

The log-likelihood is:

$$\log p_\theta(\mathbf{x}|\mathbf{z}) = \sum_{i=1}^N \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x_i - \mu_{i,\theta}(\mathbf{z}))^2}{2\sigma^2} \right] \quad (4.23)$$

$$= -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - \mu_{i,\theta}(\mathbf{z}))^2 \quad (4.24)$$

$$= -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|^2 \quad (4.25)$$

This is proportional to the negative squared error between the input $\mathbf{x}$ and the reconstructed mean $\boldsymbol{\mu}_\theta(\mathbf{z})$. The reconstruction term becomes:

$$\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[-\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|^2 \right] \quad (4.26)$$

Since the constant term doesn't affect optimization w.r.t θ, ϕ , maximizing this term is equivalent to minimizing the expected squared error: $\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|^2]$.

After clearing things up, let us go back to (4.15). There's a problem with the reconstruction term $\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\dots]$. We need to compute its gradient with respect to ϕ . However, the expectation is taken over a distribution q_ϕ whose parameters $(\boldsymbol{\mu}_\phi, \boldsymbol{\sigma}_\phi^2)$ depend on ϕ . Computationally, standard backpropagation cannot handle gradients through the sampling process $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$.

The famous reparameterization trick (Kingma and Welling [2014]) solves this (which is also applicable to VarFA). For distributions like the Gaussian, we can rewrite the sampling process. Instead of sampling $\mathbf{z}$ directly from $\mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x})))$, we first sample a noise vector $\boldsymbol{\epsilon}$ from a fixed, simple distribution, typically a standard normal: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Then we can transform this noise using the parameters from the encoder:

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon} \quad (4.27)$$

where $\odot$ denotes the element-wise product (Hadamard product), and $\boldsymbol{\sigma}_\phi(\mathbf{x})$ is the vector of standard deviations (square root of the variances $\boldsymbol{\sigma}_\phi^2(\mathbf{x})$). Note that the encoder usually outputs $\log \boldsymbol{\sigma}_\phi^2(\mathbf{x})$, so $\boldsymbol{\sigma}_\phi(\mathbf{x}) = \exp(0.5 \times \log \boldsymbol{\sigma}_\phi^2(\mathbf{x}))$.

Now, $\mathbf{z}$ still follows the desired distribution $q_\phi(\mathbf{z}|\mathbf{x})$, but the stochasticity comes from $\boldsymbol{\epsilon}$, which is independent of ϕ . The dependence on ϕ is now deterministic through $\boldsymbol{\mu}_\phi(\mathbf{x})$ and $\boldsymbol{\sigma}_\phi(\mathbf{x})$.

We can rewrite the reconstruction term in (4.15) using this trick:

$$\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} [\log p_\theta(\mathbf{x}|\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon})] \quad (4.28)$$

In practice, we approximate this expectation using Monte Carlo sampling, typically with just one sample ($L = 1$) of $\boldsymbol{\epsilon}$ per data point $\mathbf{x}$ per training step:

$$\log p_\theta(\mathbf{x}|\mathbf{z}^{(i,l)}) \quad \text{where} \quad \mathbf{z}^{(i,l)} = \boldsymbol{\mu}_\phi(\mathbf{x}^{(i)}) + \boldsymbol{\sigma}_\phi(\mathbf{x}^{(i)}) \odot \boldsymbol{\epsilon}^{(l)} \quad \text{and} \quad \boldsymbol{\epsilon}^{(l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (4.29)$$

Here we used superscript to distinguish sampled data points from general variables and coordinates. Combining everything, the objective function we want to maximize (or minimize its negative) for a single data point $\mathbf{x}$, using the reparameterization trick (4.27) and the analytical KL divergence (4.17), is approximated as:

$$\begin{aligned} \mathcal{L}(\theta, \phi; \mathbf{x}) &\approx \log p_\theta(\mathbf{x}|\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}) \\ &\quad - \frac{1}{2} \sum_{j=1}^N (\mu_{j,\phi}^2(\mathbf{x}) + \sigma_{j,\phi}^2(\mathbf{x}) - \log(\sigma_{j,\phi}^2(\mathbf{x})) - 1) \end{aligned} \quad (4.30)$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is sampled once.

We typically minimize the *negative* ELBO averaged over a mini-batch of M data points:

$$\text{Loss} = -\frac{1}{M} \sum_{i=1}^M \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) \quad (4.31)$$

$$\approx \frac{1}{M} \sum_{i=1}^M \left[\underbrace{-\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})}_{\text{Reconstruction Loss}} + \underbrace{\frac{1}{2} \sum_{j=1}^N (\mu_{j,\phi}^2(\mathbf{x}^{(i)}) + \sigma_{j,\phi}^2(\mathbf{x}^{(i)}) - \log(\sigma_{j,\phi}^2(\mathbf{x}^{(i)})) - 1)}_{\text{KL Loss}} \right] \quad (4.32)$$

where $\mathbf{z}^{(i)} = \boldsymbol{\mu}_\phi(\mathbf{x}^{(i)}) + \boldsymbol{\sigma}_\phi(\mathbf{x}^{(i)}) \odot \boldsymbol{\epsilon}^{(i)}$ with $\boldsymbol{\epsilon}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

The term $-\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})$ is the reconstruction loss (e.g., binary cross-entropy or mean squared error, depending on the data type and choice of p_θ).

At this point, we have broken down the VAE algorithm into as many details as possible for mathematical clarity. Let us end this section with the formal procedure of the training process and data generation after the training is done, so that the readers know what exactly is being computed in each step.

Training Procedure of VAE

1. Initialize encoder parameters ϕ and decoder parameters θ .
2. Repeat until convergence:
 - (a) Sample a mini-batch of data $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(M)}\}$.
 - (b) For each $\mathbf{x}^{(i)}$ in the mini-batch:
 - i. Pass $\mathbf{x}^{(i)}$ through the encoder network to get $\boldsymbol{\mu}_\phi(\mathbf{x}^{(i)})$ and $\log \sigma_\phi^2(\mathbf{x}^{(i)})$.
 - ii. Calculate $\sigma_\phi(\mathbf{x}^{(i)}) = \exp(0.5 \times \log \sigma_\phi^2(\mathbf{x}^{(i)}))$.
 - iii. Sample $\boldsymbol{\epsilon}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
 - iv. Compute the latent variable sample: $\mathbf{z}^{(i)} = \boldsymbol{\mu}_\phi(\mathbf{x}^{(i)}) + \sigma_\phi(\mathbf{x}^{(i)}) \odot \boldsymbol{\epsilon}^{(i)}$.
 - v. Pass $\mathbf{z}^{(i)}$ through the decoder network to get the parameters of $p_\theta(\mathbf{x}|\mathbf{z}^{(i)})$ (e.g., $\mathbf{p}_\theta(\mathbf{z}^{(i)})$ for Bernoulli, or $\boldsymbol{\mu}_\theta(\mathbf{z}^{(i)})$ for Gaussian).
 - vi. Calculate the reconstruction loss term $-\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})$.
 - vii. Calculate the KL divergence term $D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}^{(i)}) || p(\mathbf{z}))$ using the analytical formula (4.17) with $\boldsymbol{\mu}_\phi(\mathbf{x}^{(i)})$ and $\sigma_\phi^2(\mathbf{x}^{(i)})$.
 - (c) Compute the average loss over the mini-batch (Negative ELBO, (4.32)).
 - (d) Compute gradients of the loss with respect to ϕ and θ using back propagation (enabled by the reparameterization trick).
 - (e) Update ϕ and θ using an optimizer (e.g., Adam, SGD).

Data Generation Procedure after Training

Once the VAE is trained, we can generate new data points $\mathbf{x}_{new}$ that resemble the training data:

1. Sample a latent vector $\mathbf{z}_{new}$ from the prior distribution: $\mathbf{z}_{new} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$.
2. Pass $\mathbf{z}_{new}$ through the trained decoder network to obtain the parameters of $p_\theta(\mathbf{x}|\mathbf{z}_{new})$.
3. Sample a new data point $\mathbf{x}_{new}$ from this distribution $p_\theta(\mathbf{x}|\mathbf{z}_{new})$. (For example, if p_θ is Gaussian with mean $\boldsymbol{\mu}_\theta(\mathbf{z}_{new})$ and fixed variance σ^2 , then $\mathbf{x}_{new} \sim \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{z}_{new}), \sigma^2 \mathbf{I})$). Often, for image generation, people just use the mean output of the decoder (e.g., $\mathbf{p}_\theta(\mathbf{z}_{new})$ or $\boldsymbol{\mu}_\theta(\mathbf{z}_{new})$) directly as the generated sample, rather than sampling from the output distribution.

4.4 The High-Level View of VarFA based on VAE

After going through VAE thoroughly, we can end the introduction of chapter 4 with a short section highlighting the difference between VarFA and VAE. The mathematical deduction in the previous section, along with the high-level description provided in this section should prepare us well for the content in chapter 4. Generally speaking, VarFA can be applied in all scenarios where both dimension reduction and interoperability are necessary. In our case,

we choose education as the main field of application because the necessity and advantage of interpretable models and dimension reduction are more or less self-evident.

In our work, VarFA starts with two more interpretable assumptions compared to VAE:

1. The students' overall performance follow a FA model.
2. The students' latent skills follow a multi-dimensional Gaussian distribution. (Like we mentioned previously, we assume everyone performs around their average level, with over low and over high performance being exponentially rare.)

With these two assumptions, VarFA uses a similar encoder-decoder architecture as VAE, and VarFA can also be trained using the reparameterization trick efficiently based on a modified ELBO (details in chapter 4). After the training is complete, VarFA takes students' performance record and determines their latent skills during the encoder phase. This step is much more interpretable than the usual VAE and much more similar to the natural thinking process of humans. For instance, if a student performs well in math and physics, VarFA will assign a high competence factor for their quantitative reasoning skills. This is crucial for machine learning in education, as prediction is not the end goal, but the reflection and feedback to students.

In the decoder phase, given students' latent skills, questions' difficulty and their correlations to the skills, VarFA is able to predict students' performance in a natural, interpretable way. Intuitively speaking, if a student is good at reading and writing, it is very likely for them to answer an easy question in the subject of literature. We can use VarFA to track the performance of students and predict their potential areas for improvement. On the other hand, VarFA can also be used to spot possible academic dishonesty. A well-trained VarFA model can detect students who suddenly perform well in a subject they have been struggling with. This interpretable generative model is a versatile and useful tool to use in academic environments.

VarFA marks the small but meaningful effort to transfer the power of machine learning into human learning. As machines learn faster and better, we should not forget our shared responsibility to prepare our future generations for this ever-changing world. Because the new science and technology require a longer study plan than ever, it is imperative to make sure that we use the advancements in AI to make sure that knowledge is widely accessible, and we are indeed making this world a better place.

References

- [1] Tony Hey, Stewart Tansley, Kristin Tolle, and Jim Gray. *The Fourth Paradigm: Data-Intensive Scientific Discovery*. Microsoft Research, October 2009. ISBN 978-0-9825442-0-4. URL <https://www.microsoft.com/en-us/research/publication/fourth-paradigm-data-intensive-scientific-discovery/>.
- [2] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012. URL https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [4] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(2579-2605), 2008.
- [5] David Donoho. High-dimensional data analysis: The curses and blessings of dimensionality. *AMS Math Challenges Lecture*, pages 1–32, 01 2000.
- [6] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. https://academic.oup.com/jrsssb/article-pdf/58/1/267/49098631/jrsssb_58_1_267.pdf. URL <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>.
- [7] Shaobing Chen and D. Donoho. Basis pursuit. In *Proceedings of 1994 28th Asilomar Conference on Signals, Systems and Computers*, volume 1, pages 41–44 vol.1, 1994.
- [8] Jianqing Fan and Jinchi Lv. A selective overview of variable selection in high dimensional feature space. *Statistica Sinica*, 20:101–148, 01 2010.
- [9] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *ICLR*, 2014.
- [10] Richard Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961. ISBN 9780691079011. URL <http://www.jstor.org/stable/j.ctt183ph6v>.
- [11] G. Hughes. On the mean accuracy of statistical pattern recognizers. *IEEE Transactions on Information Theory*, 14(1):55–63, 1968.
- [12] Oliver Stegle, Sarah Teichmann, and John Marioni. Computational and analytical challenges in single-cell transcriptomics. *Nature reviews. Genetics*, 16, 01 2015.
- [13] Vladimir Kiselev, Tallulah Andrews, and Martin Hemberg. Challenges in unsupervised clustering of single-cell rna-seq data. *Nature Reviews Genetics*, 20:1, 01 2019.
- [14] Peter Visscher, Naomi Wray, Qian Zhang, Pamela Sklar, Mark McCarthy, Matthew Brown, and Jian Yang. 10 years of gwas discovery: Biology, function, and translation. *The American Journal of Human Genetics*, 101:5–22, 07 2017.
- [15] F.R.S Karl Pearson. Liii. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2 (11):559–572, 1901, <https://doi.org/10.1080/14786440109462720>. URL <https://doi.org/10.1080/14786440109462720>.

- [16] Harold Hotelling. Analysis of a complex of statistical variables into principal components. *J. Ed. Psych.*, 24:417–441, 1933.
- [17] W. S. Torgerson. Multidimensional scaling: I. theory and method. *Psychometrika*, 17: 401–419, 1952.
- [18] W.S. Torgerson. *Theory and methods of scaling*. Wiley, 1958. URL <https://books.google.com/books?id=FWN-AAAAMAAJ>.
- [19] Roger N. Shepard. The analysis of proximities: Multidimensional scaling with an unknown distance function. i. and ii. *Psychometrika*, 27(2):125–140, 1962.
- [20] J. B. Kruskal. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika*, 29(1):1–27, 1964.
- [21] Sam T. Roweis and Lawrence K. Saul. Nonlinear Dimensionality Reduction by Locally Linear Embedding. *Science*, 290(5500):2323–2326, 2000, <http://www.sciencemag.org/cgi/reprint/290/5500/2323.pdf>. URL <http://www.sciencemag.org/cgi/content/abstract/290/5500/2323>.
- [22] Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319, 2000.
- [23] M.L. Fredman and R.E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. In *25th Annual Symposium on Foundations of Computer Science, 1984.*, pages 338–346, 1984.
- [24] Geoffrey E Hinton and Sam Roweis. Stochastic neighbor embedding. In S. Becker, S. Thrun, and K. Obermayer, editors, *Advances in Neural Information Processing Systems*, volume 15. MIT Press, 2002. URL https://proceedings.neurips.cc/paper_files/paper/2002/file/6150ccc6069bea6b5716254057a194ef-Paper.pdf.
- [25] Antonio Auffinger and Daniel Fletcher. Equilibrium distributions for t-distributed stochastic neighbour embedding, 2023, 2304.03727. URL <https://arxiv.org/abs/2304.03727>.
- [26] Dmitry Kobak, George Linderman, Stefan Steinerberger, Yuval Kluger, and Philipp Berens. *Heavy-Tailed Kernels Reveal a Finer Cluster Structure in t-SNE Visualisations*, page 124–139. Springer International Publishing, 2020. ISBN 9783030461508. URL http://dx.doi.org/10.1007/978-3-030-46150-8_8.
- [27] Ben Weinkove. Dimension reduction and the gradient flow of relative entropy, 2024, 2409.16963. URL <https://arxiv.org/abs/2409.16963>.
- [28] Stuart Geman and Donald Geman. Stochastic relaxation, gibbs distributions, and the bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-6(6):721–741, 1984.
- [29] Alex Zihao Zhu, Jieru Mei, Siyuan Qiao, Hang Yan, Yukun Zhu, Liang-Chieh Chen, and Henrik Kretschmar. Superpixel transformers for efficient semantic segmentation, 2023, 2309.16889. URL <https://arxiv.org/abs/2309.16889>.
- [30] Y.Y. Boykov and M.-P. Jolly. Interactive graph cuts for optimal boundary and region segmentation of objects in n-d images. In *Proceedings Eighth IEEE International Conference on Computer Vision. ICCV 2001*, volume 1, pages 105–112 vol.1, 2001.
- [31] V. Kolmogorov and R. Zabini. What energy functions can be minimized via graph cuts? *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(2):147–159, 2004.

- [32] Y. Boykov and V. Kolmogorov. An experimental comparison of min-cut/max- flow algorithms for energy minimization in vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(9):1124–1137, 2004.
- [33] Magali Naville and Daniel Gautheret. Transcription attenuation in bacteria: theme and variations. *Briefings in Functional Genomics*, 9:178–189, 03 2010.
- [34] François Jacob and Jacques Monod. Genetic regulatory mechanisms in the synthesis of proteins. *Journal of Molecular Biology*, 3(3):318–356, 1961. ISSN 0022-2836. URL <https://www.sciencedirect.com/science/article/pii/S0022283661800727>.
- [35] Jonna Frasor, Fabio Stossi, Jeanne M. Danes, Barry Komm, C. Richard Lyttle, and Benita S. Katzenellenbogen. Selective estrogen receptor modulators: Discrimination of agonistic versus antagonistic activities by gene expression profiling in breast cancer cells. *Cancer Research*, 64(4):1522–1533, 02 2004, <https://aacrjournals.org/cancerres/article-pdf/64/4/1522/2523009/zch00404001522.pdf>. ISSN 0008-5472. URL <https://doi.org/10.1158/0008-5472.CAN-03-3326>.
- [36] Nina Heldring, Ashley Pike, Sandra Andersson, Jason Matthews, Guojun Cheng, Johan Hartman, Michel Tujague, Anders Ström, Eckardt Treuter, Margaret Warner, and Jan-Åke Gustafsson. Estrogen receptors: How do they signal and what are their targets. *Physiological Reviews*, 87(3):905–931, 2007, <https://doi.org/10.1152/physrev.00026.2006>. URL <https://doi.org/10.1152/physrev.00026.2006>. PMID: 17615392.
- [37] Nir Friedman, Michal Linial, Iftach Nachman, and Dana Pe’er. Using bayesian networks to analyze expression data. *Journal of Computational Biology*, 7(3-4):601–620, 2000, <https://doi.org/10.1089/106652700750050961>. URL <https://doi.org/10.1089/106652700750050961>. PMID: 11108481.
- [38] Karen Sachs, Omar Perez, Dana Pe’er, Douglas Lauffenburger, and Garry Nolan. Causal protein-signaling networks derived from multiparameter single-cell data. *Science (New York, N.Y.)*, 308:523–9, 05 2005.
- [39] A Krogh. Hidden markov models in computational biology. applications to protein modeling. *Journal of molecular biology.*, 235(5), 1994-01-31. ISSN 0022-2836.
- [40] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, 1998.
- [41] Alexander Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*, volume B. Springer Science and Business Media, 01 2003.
- [42] JAMES E. LIGHTNER. A brief look at the history of probability and statistics. *The Mathematics Teacher*, 84(8):623–630, 1991. ISSN 00255769. URL <http://www.jstor.org/stable/27967334>.
- [43] Leonhard Euler. Solutio problematis ad geometriam situs pertinentis. *Commentarii Academiae Scientiarum Imperialis Petropolitanae*, 8:128–140, 1736.
- [44] Ernst Ising. Beitrag zur theorie des ferromagnetismus. *Zeitschrift für Physik*, 31:253–258, 1925. URL <https://api.semanticscholar.org/CorpusID:122157319>.
- [45] Lars Onsager. Crystal statistics. i. a two-dimensional model with an order-disorder transition. *Phys. Rev.*, 65:117–149, Feb 1944. URL <https://link.aps.org/doi/10.1103/PhysRev.65.117>.
- [46] Sewall Wright. Correlation and causation. *Journal of agricultural research*, 20(7):557–585, 1921.

- [47] Sewall Wright. The method of path coefficients. *The Annals of Mathematical Statistics*, 5(3):161–215, 1934. ISSN 00034851, 21688990. URL <http://www.jstor.org/stable/2957502>.
- [48] Sewall Wright. Systems of mating. i. the biometric relations between parent and offspring. *Genetics*, 6(2):111–123, 03 1921, <https://academic.oup.com/genetics/article-pdf/6/2/111/35397881/genetics0111.pdf>. ISSN 1943-2631. URL <https://doi.org/10.1093/genetics/6.2.111>.
- [49] Sewall Wright. The relative importance of heredity and environment in determining the piebald pattern of guinea-pigs. *Proceedings of the National Academy of Sciences of the United States of America*, 6 6:320–32, 1920. URL <https://api.semanticscholar.org/CorpusID:12577660>.
- [50] Sewall Wright. On the nature of size factors. *Genetics*, 3(4):367–374, 07 1918, <https://academic.oup.com/genetics/article-pdf/3/4/367/35081662/genetics0367.pdf>. ISSN 1943-2631. URL <https://doi.org/10.1093/genetics/3.4.367>.
- [51] Otis Dudley Duncan. *Introduction to Structural Equation Models*. Academic Press, 1975.
- [52] Kenneth A. Bollen. *Structural Equations with Latent Variables*. Wiley, 1989. ISBN 9781118619179.
- [53] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1988. ISBN 1558604790.
- [54] Mehran Sahami, Susan Dumais, David Heckerman, and Eric Horvitz. A bayesian approach to filtering junk e-mail. *AAAI’98 Workshop on Learning for Text Categorization*, 62, 06 1998.
- [55] Catherine Shenoy and Prakash Shenoy. *Bayesian Network Models of Portfolio Risk and Return*, pages 87–. The MIT Press, 06 2003.
- [56] Jozef Gemela. Financial analysis using bayesian networks. *Applied Stochastic Models in Business and Industry*, 17:57 – 67, 02 2001.
- [57] Julian Besag. Spatial interaction and the statistical analysis of lattice systems. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):192–225, 1 1974, https://academic.oup.com/jrsssb/article-pdf/36/2/192/49096627/jrsssb_36_2.192.pdf. ISSN 0035-9246. URL <https://doi.org/10.1111/j.2517-6161.1974.tb00999.x>.
- [58] Julian Besag. On the statistical analysis of dirty pictures. *Journal of the Royal Statistical Society: Series B (Methodological)*, 48(3):259–279, 07 1986, https://academic.oup.com/jrsssb/article-pdf/48/3/259/49097842/jrsssb_48_3.259.pdf. ISSN 0035-9246. URL <https://doi.org/10.1111/j.2517-6161.1986.tb01412.x>.
- [59] F.R. Kschischang, B.J. Frey, and H.-A. Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.
- [60] Michael Jordan, Zoubin Ghahramani, Tommi Jaakkola, and Lawrence Saul. An introduction to variational methods for graphical models. *Machine Learning*, 37:183–233, 01 1999.
- [61] Alan E. Gelfand and Adrian F. M. Smith and. Sampling-based approaches to calculating marginal densities. *Journal of the American Statistical Association*, 85(410):398–409, 1990, <https://www.tandfonline.com/doi/pdf/10.1080/01621459.1990.10476213>. URL <https://www.tandfonline.com/doi/abs/10.1080/01621459.1990.10476213>.

- [62] D. Koller and N. Friedman. *Probabilistic Graphical Models: Principles and Techniques*. Adaptive computation and machine learning. MIT Press, 2009. ISBN 9780262013192. URL <https://books.google.co.in/books?id=7dzpHCHzNQ4C>.
- [63] Paul Erdős and Lovász László. Problems and results on 3-chromatic hypergraphs and some related questions. *Coll Math Soc J Bolyai*, 10, 01 1974.
- [64] James B Shearer. On a problem of spencer. *Combinatorica*, 5(3):241–245, July 1985. ISSN 0209-9683. URL <https://doi.org/10.1007/BF02579368>.
- [65] Joel Spencer. Asymptotic lower bounds for ramsey functions. *Discrete Mathematics*, 20:69–76, 1977. ISSN 0012-365X. URL <https://www.sciencedirect.com/science/article/pii/0012365X77900449>.
- [66] Robin A. Moser and Gábor Tardos. A constructive proof of the general lovasz local lemma, 2009, 0903.0544. URL <https://arxiv.org/abs/0903.0544>.
- [67] Hugo Mak, Runze Han, and Hoover Yin. Application of variational autoencoder (vae) model and image processing approaches in game design, 02 2023.
- [68] C. Spearman. General intelligence, objectively determined and measured. *The American Journal of Psychology*, 15(2):201–292, 1904. ISSN 00029556. URL <http://www.jstor.org/stable/1412107>.
- [69] L.L. Thurstone. *Multiple-factor Analysis: A Development and Expansion of The Vectors of the Mind*. University of Chicago Committee on Publications in Biology and Medicine. Publications. University of Chicago Press, 1947. ISBN 9780608111896. URL <https://books.google.com/books?id=c0wAAAAAAAJ>.
- [70] K. G. Jöreskog. Some contributions to maximum likelihood factor analysis. *Psychometrika*, 32(4):443–482, 1967.
- [71] Karl Jöreskog. A general approach to confirmatory factor analysis. *Psychometrika*, 34: 183–202, 02 1969.
- [72] Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning, 2017, 1702.08608. URL <https://arxiv.org/abs/1702.08608>.
- [73] Tim Miller. Explanation in artificial intelligence: Insights from the social sciences, 2018, 1706.07269. URL <https://arxiv.org/abs/1706.07269>.
- [74] Carl Doersch. Tutorial on variational autoencoders, 2021, 1606.05908. URL <https://arxiv.org/abs/1606.05908>.